SDLC-HARNESS · DISCOVERY

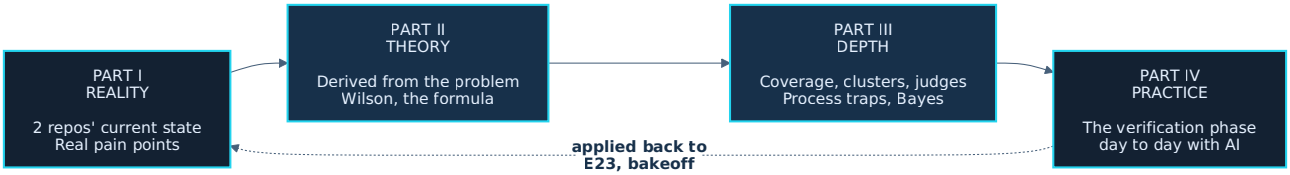
Confidence Intervals for Probe-First

From two repos' reality to verification methodology — and back to daily practice when working with AI

Date	2026-08-04
Repos	vsf/sdlc-harness · address_text_scan_dongpv
Evidence	E11, E14, E23, E25 · artifact-bakeoff-verdict
Numbers	computed with python3 + statistics.NormalDist
Status	discovery — no plan yet

Where this document goes

Four parts, one full circle: start from **what is actually happening** in two repos, derive the theory **from that problem**, dig to the bottom, then return to **daily operations**.



How to read by role:

You are	Read
Running probes daily	Part I → Part IV (skip the math)
Designing gates for the harness	Part I → III.5 → Part IV
Wanting to understand why	Read all four in order
About to write `wilson.py`	Part II → III → IV.7

> **A note on the numbers.** Every number in this document was computed with `python3` + `statistics.NormalDist` during the 2026-08-04 session. Coverage figures are computed **exactly** over the binomial distribution — summed across every possible value of `k`, not simulated, so they carry no random error. Sections that do use simulation state their `seed`. Numbers quoted from repo evidence are kept verbatim, with the source cited.

PART I

REALITY

What is actually happening in two repos. Not hypothetical examples — real numbers, real files, and the places currently drifting that nobody has named yet.

I.1 — The familiar scene

You run 10 probes. 8 pass. You write in the report:

```
[OBSERVED] pipeline achieves 80% accuracy
```

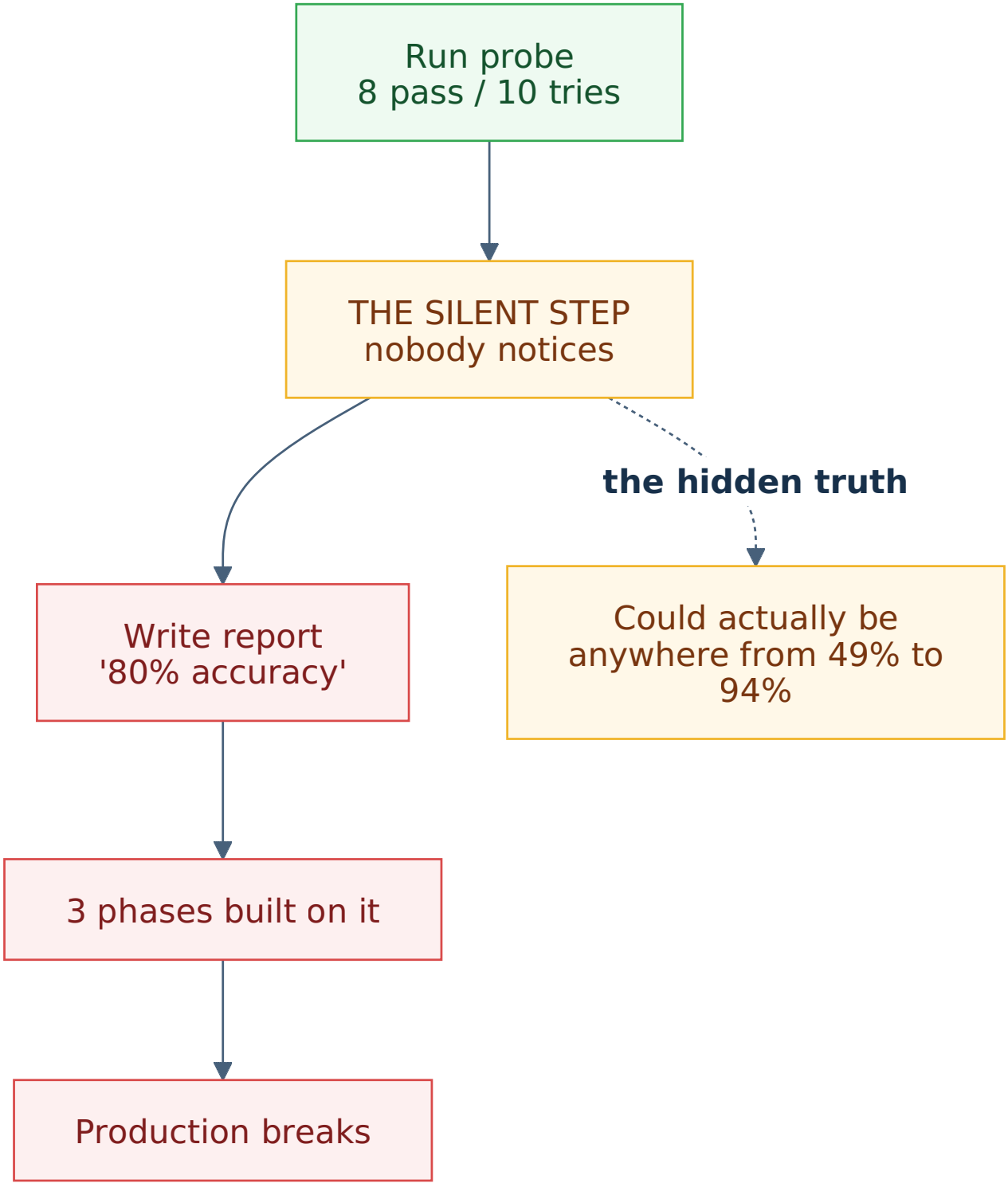
Three weeks later, three phases have been built on that line. Production breaks. Someone digs for the cause and finds that report line — it carries the `[OBSERVED]` label, meaning it was actually run, actually observed. Nobody lied.

So where did it break?

The break is not in the probe

The probe was correct. 8 out of 10 is the truth.

The break is in the **step from "8/10" to "80% accuracy"**. That step feels so obvious that nobody thinks of it as a step. But it is one, and it has no basis.



With 10 trials, the truth could sit anywhere from **49% to 94%**. Your pipeline might be correct only half the time, and the data you hold **cannot rule that out**.

The number 80% is not wrong. It merely **hides the most important thing**: you do not know it is 80%.

Four ways intuition fails

You see	You think	The truth (95% interval)
3/3 pass	"100%, certain"	could be as low as **43.9%**
10/10 pass	"perfect"	could be as low as **72.2%**
8/10 pass	"80%, pretty good"	from **49.0%** to 94.3%
0/30 errors	"no errors"	error rate could reach **11.4%**

These four lines are four ways probe-first shoots itself in the foot. None of them are **the prober's fault** — they are where human intuition breaks.

I.2 — Why this hurts probe-first specifically

The harness's probe-first principle says: *run the real thing before building on a guess*. That principle is correct. But it produces a recurring situation.

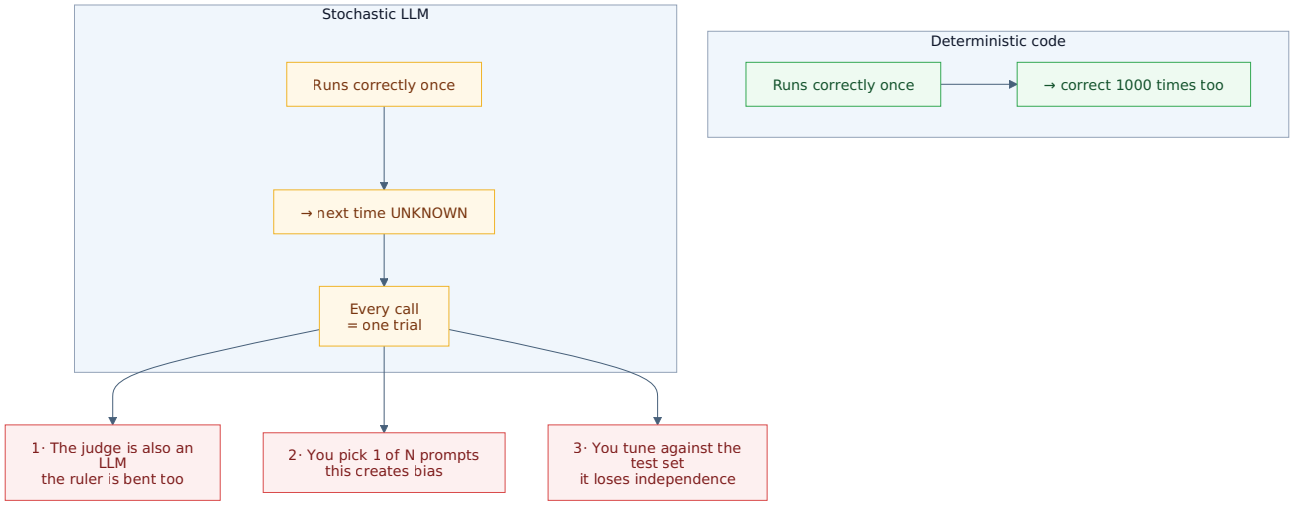
Probes are small. You run 5, 10, 30 samples — not 5000. Rendering 100 PDF pages and calling an LLM on each costs real time and money. E14 measured **23.8 seconds per page** — 100 pages is nearly 40 minutes for a single pass.

And the small-sample regime is exactly where every simple formula breaks.

Three things that only happen with LLMs

Ordinary code is **deterministic**: if it runs correctly once, it runs correctly 1000 times. If `parse_date()` works once, you do not need to run it 100 more times.

LLMs are not. Same prompt, same input, different results. Every LLM call is **a random trial**, and one successful run **proves nothing** about the next.



Those three branches are three problems Part III will dig into — and all three **exist right now** in the OCR repo.

I.3 — Current state, OCR repo: `address_text_scan_dongpv`

This repo processes scanned images of Vietnamese Land Use Right Certificates (GCN QSDĐ). It holds **159 evidence files**, numbered E11 through E45.

And it works **very carefully**. That is the most important thing to say first.

I.3.1. E11 — they built their own ruler

E11 spotted a gap many projects never see:

> Every TRUE/FALSE signal is currently scored against the current software's output — that is, "what the machine says now", NOT "the truth on the paper".
>
> — `docs/evidences/E11-260803-bang-dap-an-chuan.md`

That document's own analogy: "*Like measuring a table with a wooden ruler nobody has checked for straightness.*"

So they built a real ruler: opened every one of 100 pages, looked with their own eyes, recorded the truth. And they **forbade** the labellers from reading the software's output — because "*if they see what the software says, the ruler will bend to match the software*".

This is a **golden set** in the proper sense, built the proper way.

The first pass was off by one page, and they **caught it, recorded it, and redid it**.

I.3.2. E14 — they avoided the "meaningless 100%" trap

E14 tried Claude Haiku for guessing page orientation on scans. The real dataset has **all 100 pages upright**.

The E14 document states plainly:

> If we only measured "guess accuracy" on it, then a program that **only ever prints 0** would also score 100%. A pretty number with zero meaning.

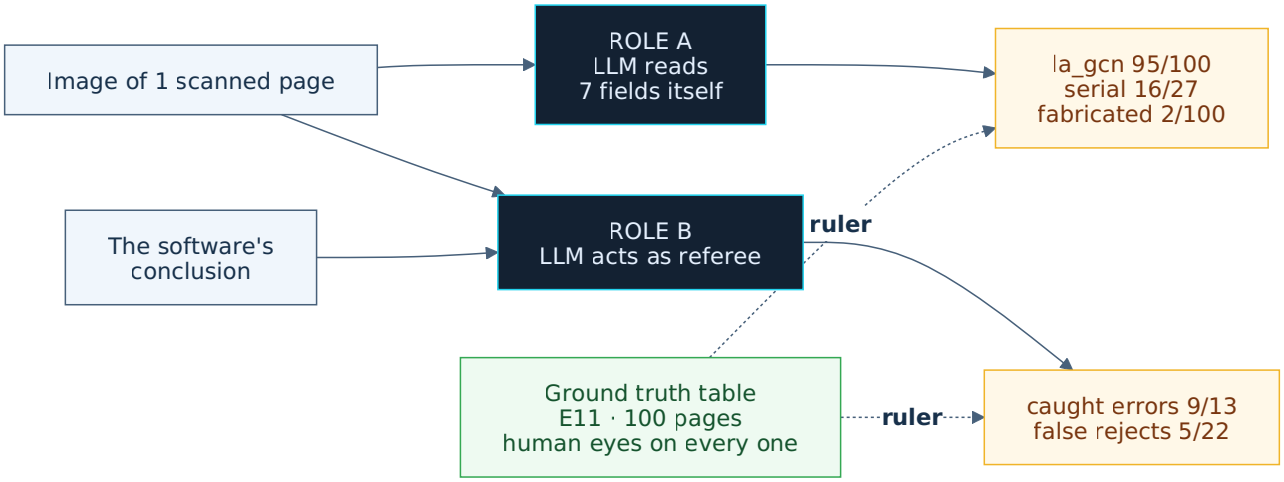
And they point out the project had already fallen into exactly that trap at E09 — Tesseract scored "13/19 correct" while answering `0` for every page.

Their fix: manually rotate 20 pages on a balanced schedule, exactly 5 pages per angle, `seed=42`.

This is precisely the **balanced calibration set** principle that Part III.3 derives mathematically — E14 got it right by instinct.

I.3.3. E23 — an LLM judge, real, with numbers

E23 is the case closest to this document's subject. Two roles:



Real numbers from `E23-260803-llm-doc-truong-va-trong-tai.md`:

Metric	Value	Interval E23 reported
`la_gcn` correct	95/100	88.8% – 97.8%
Serial fabrication rate	2/100	0.55% – 7.00%
Referee caught errors	9/13	42.4% – 87.3%
Referee false rejects	5/22	10.1% – 43.4%

> **E23 already used Wilson, and used it correctly.** I recomputed all four rows: `95/100 → [0.8882, 0.9785]`, `2/100 → [0.0055, 0.0700]`, `9/13 → [0.4237, 0.8732]`, `5/22 → [0.1012, 0.4344]`. Fully consistent with the E23 table.
>
> This repo is **not** at the starting line. It is already well down the road.

E23 even corrected one of its own scoring mistakes:

> **Correction to the first scoring pass:** in the first pass I wrote "LLM falsely rejected 31%". That number is **wrong**, due to my own scoring error, not the LLM's. […] this is the fourth time in this project I have mis-scored the ruler.

That level of honesty is rare.

I.4 — Three places still drifting

This is the uncomfortable part. The repo works very carefully, already uses Wilson, already built a golden set — and there are still **three measurable drifts**.

To be clear up front: this is **not a criticism of E23**. It is evidence that the traps in this document are real, and that they slip past even careful practitioners.

I.4.1. Drift one — 100 pages, but only 6 case files

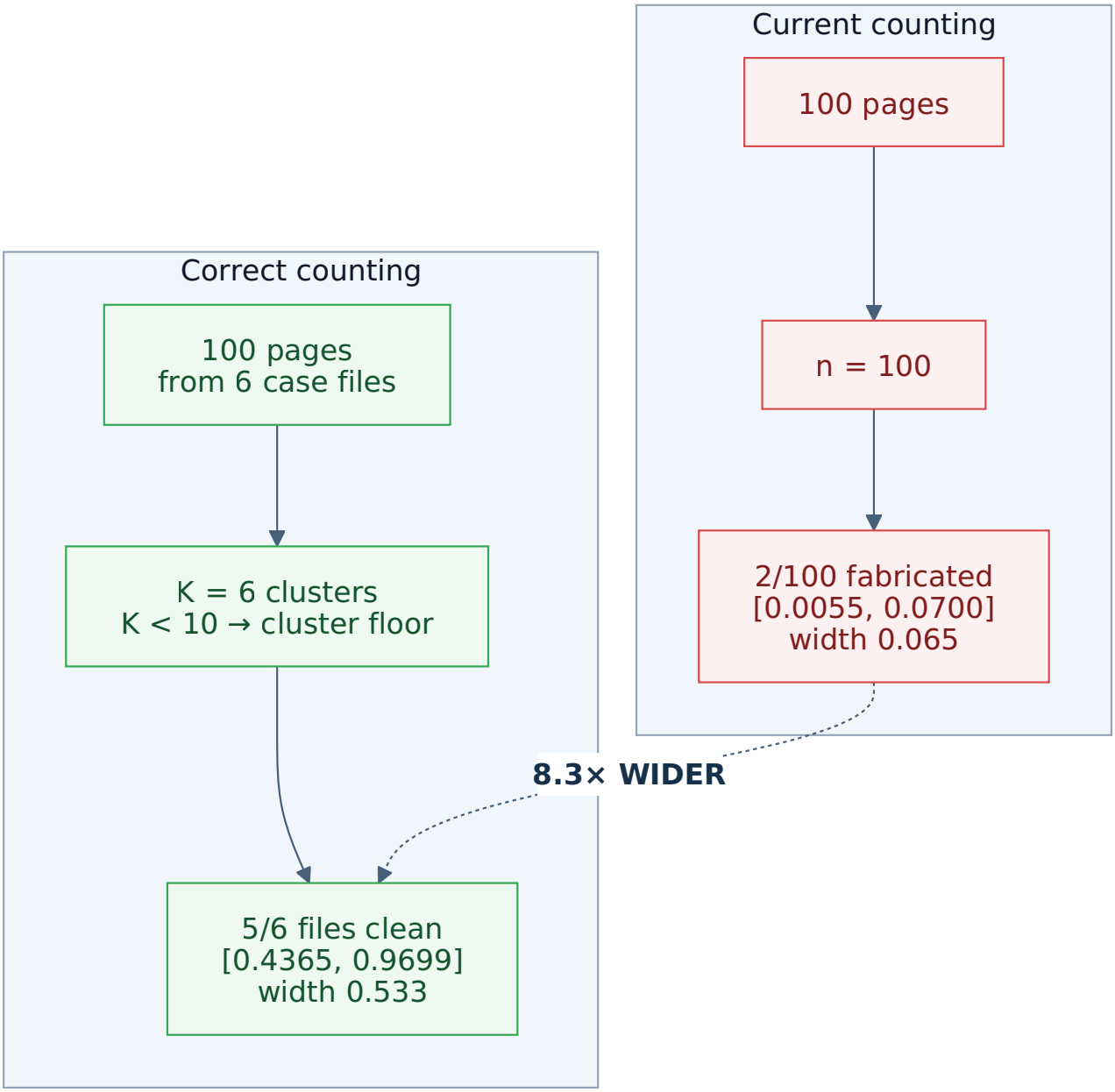
E23 ran on 100 pages. But those 100 pages come from **6 case files**: `AA00575390`, `CV234036`, `CV234050`, `CV234051`, `CV234057`, and one more.

Pages within the same case file share a scanner, paper quality, operator, and form type. They are **not independent**.

And the evidence sits inside E23 itself:

> **Distinct case files with fabrication: 1 / 6** (`AA00575390`)

Both fabrication events landed in **the same case file**. If the 100 pages were truly independent, two rare errors landing in one file would be very unlikely. This is a clear **clustering** signal.



> **The true interval is 8.3× wider than the reported one.** Not because E23 computed Wilson incorrectly — they computed it correctly. But because `n` is not 100.

The design consequence is heavier still. With 6 case files there is a **hard ceiling** you cannot cross:

Clustering level	Max effective sample, however many pages you scan
ICC = 0.1	**60.0**
ICC = 0.2	**30.0**
ICC = 0.3	**20.0**
ICC = 0.5	**12.0**

Meaning: scanning another 1000 pages from these 6 files **helps nothing**. Progress requires **more case files**, not more pages.

This is immediately actionable, and it inverts the intuition of "we need more data".

I.4.2. Drift two — the judge has not passed a quality gate

From E23's table 4.2, the referee LLM's **confusion matrix** can be reconstructed:

```
                    software WRONG    software RIGHT
LLM says WRONG          TP = 9            FP = 5
LLM says RIGHT          FN = 4            TN = 17

sens = 9/13  = 0.6923      Wilson [0.4237, 0.8732]
spec = 17/22 = 0.7727      Wilson [0.5656, 0.8988]
```

Applying the quality gate Part III.3 derives:

Criterion	Value	Result
Youden J = sens + spec − 1 ≥ 0.5	**0.4650**	**FAILS**
|sens − spec| ≤ 0.15	0.0804	passes
n_calib ≥ 50	35	fails
Error amplification factor 1/J	**2.15**	—

> **E23's judge fails the gate: J = 0.4650 < 0.5.**
>
> Meaning that if used for back-correction, every error is **multiplied by 2.15**. The resulting number would be worse than not correcting at all.
>
> Verification: using this judge to score 100 new pages yielding 88 passes, the combined interval is `[0.0000, 1.0000]` — **completely uninformative**.

This does **not** mean Role B is useless. E23 concludes correctly that it is an **alarm bell** — it caught 9 of 13 real errors, including all 5 serial errors that previously required manual hunting, and two cases where the software fabricated a number that did not exist.

The meaning is: it works to **summon a human**, not to **replace a human scorer**. Part III.3 calls this a tier-1 judge (screening), not tier-2 (correction).

I.4.3. Drift three — no language for "how much is enough"

E14 concluded Haiku guessed orientation "correctly 16/20 times on the test set". The natural next question: **is 16/20 enough to decide?**

```
16/20  →  Wilson 95% = [0.5895, 0.9208]
```

From 59% to 92%. That is the gap between "so-so" and "very good" — and 20 samples cannot tell them apart.

The repo currently has no tool to answer "how many more samples do I need". Part II.6 supplies the lookup table.

I.5 — Current state, harness repo: `vsf/sdlc-harness`

The harness has a different problem — not missing numbers, but **numbers without a basis**.

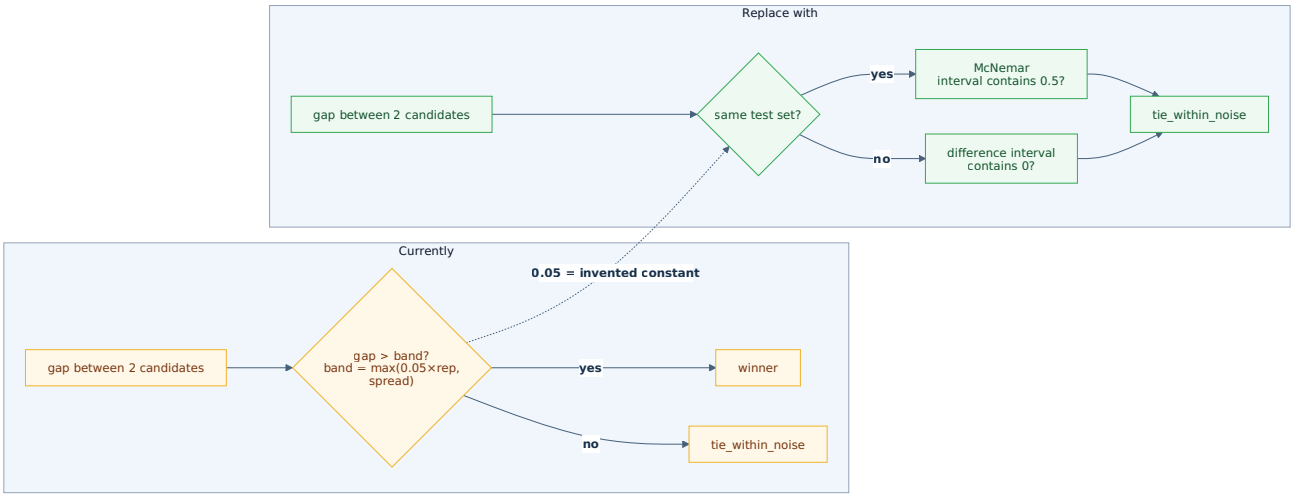
I.5.1. `hs:bakeoff` — a noise band made of a constant

The `hs:bakeoff` skill compares candidates on real measurements and already has a "tie within noise" concept. The `artifact-bakeoff-verdict.json` schema defines:

Field	Schema description
`verdict`	`winner` = beats runner-up beyond the noise band; `tie_within_noise`
`rel_band`	relative band fraction, **default 0.05**
`observed_spread_floor`	max spread across candidates — the honesty floor on band
`band`	`max(rel_band × |rep(best)|, observed_spread_floor)`
`gap`	improvement of best over runner-up (≥ 0)

The design is already good: it has a noise concept, an honesty floor, and no silent truncation.

But **`rel_band = 0.05` is an invented constant**. Why 5%? Nothing in the data says so. At small `n` the real noise is far larger than 5%; at large `n`, 5% is far too wide.



Part III.4 supplies the replacement — a noise band computed **from the data itself**, not from a constant.

I.5.2. The `[OBSERVED]` label is not tight enough for rates

The harness has three labels: `[OBSERVED]` (actually run), `[ASSUMED]` (unverified), `[PRIOR]` (training knowledge not re-checked).

These three answer **"was it actually run"**. They do not answer **"how many times"**.

`[OBSERVED] 3/3 pass` and `[OBSERVED] 300/300 pass` carry the same label, but one guarantees ≥43.9% and the other ≥98.9%.

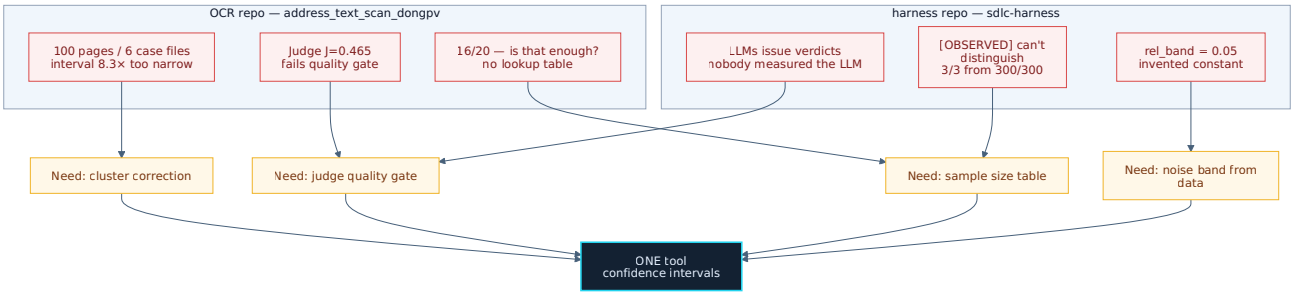
I.5.3. No mandatory procedure for LLM scoring

The harness uses LLMs at many points: `hs:code-review`, `hs:critique`, `hs:test`, and the `hs:red-teamer` / `hs:independent-revalidator` agents. Several of those **produce verdicts**.

There is currently no requirement to measure **the very LLM issuing the verdict** before trusting it.

Meaning: gates are measuring the pipeline **through an uncalibrated scale**, and nobody knows how far off the scale is. E23 shows it can be off enough to fail the gate.

I.6 — Pain points, consolidated



Six pain points, four needs, and all four reduce to **one tool**: a way to say "with this much data, how far am I entitled to trust it".

Part II derives that tool from the problems above.

I.7 — What this document does NOT promise

Stated up front to avoid false expectations.

The tools here address **one** kind of uncertainty: uncertainty from **small samples**. They do not address:

- **Biased samples** — a golden set of only clean images tells you only about clean images. See III.5.6.
- **Process errors** — stopping when it looks good, picking the luckiest candidate. See III.5.
- **Not thinking** — pasting an interval into a report and calling it rigour.

The last is the biggest risk, and Part IV.9 returns to it.

PART II

THEORY

Deriving the tool from the six pain points in Part I — not as detached knowledge, but as the answer to the problem just stated.

II.1 — The idea, via a pot of soup

You taste 3 spoonfuls from a pot of soup. All 3 are salty.

What do you conclude? Not "the pot is 100% salty". You conclude: "*probably salty, but I have only tasted 3 spoonfuls*".

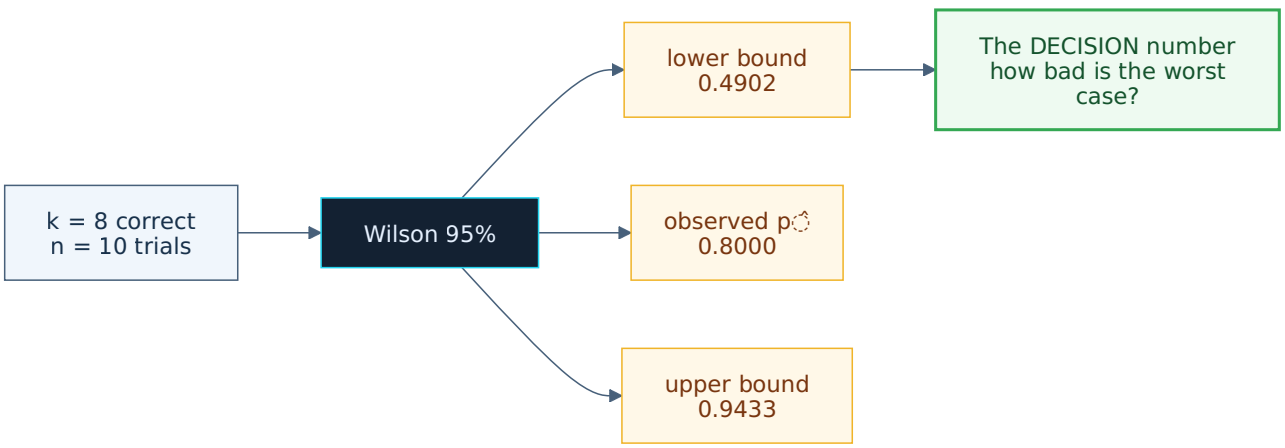
A **confidence interval** is that second clause — "*but I have only tasted 3 spoonfuls*" — expressed as a number. It turns `3/3` into `[0.4385, 1.0000]`: could be great, could also be that barely half the pot is salty, and your 3 spoonfuls cannot tell those apart.

Three numbers, not one

The naive calculation gives **one** number:

```
p̂ = k / n          (p̂ reads "p-hat" — the observed rate)
8/10 → p̂ = 0.80
```

A confidence interval gives **three**:



The number used **to decide** is almost always the **lower bound**. It answers: "*how bad is the worst case?*" — and that is the question when you are about to ship.

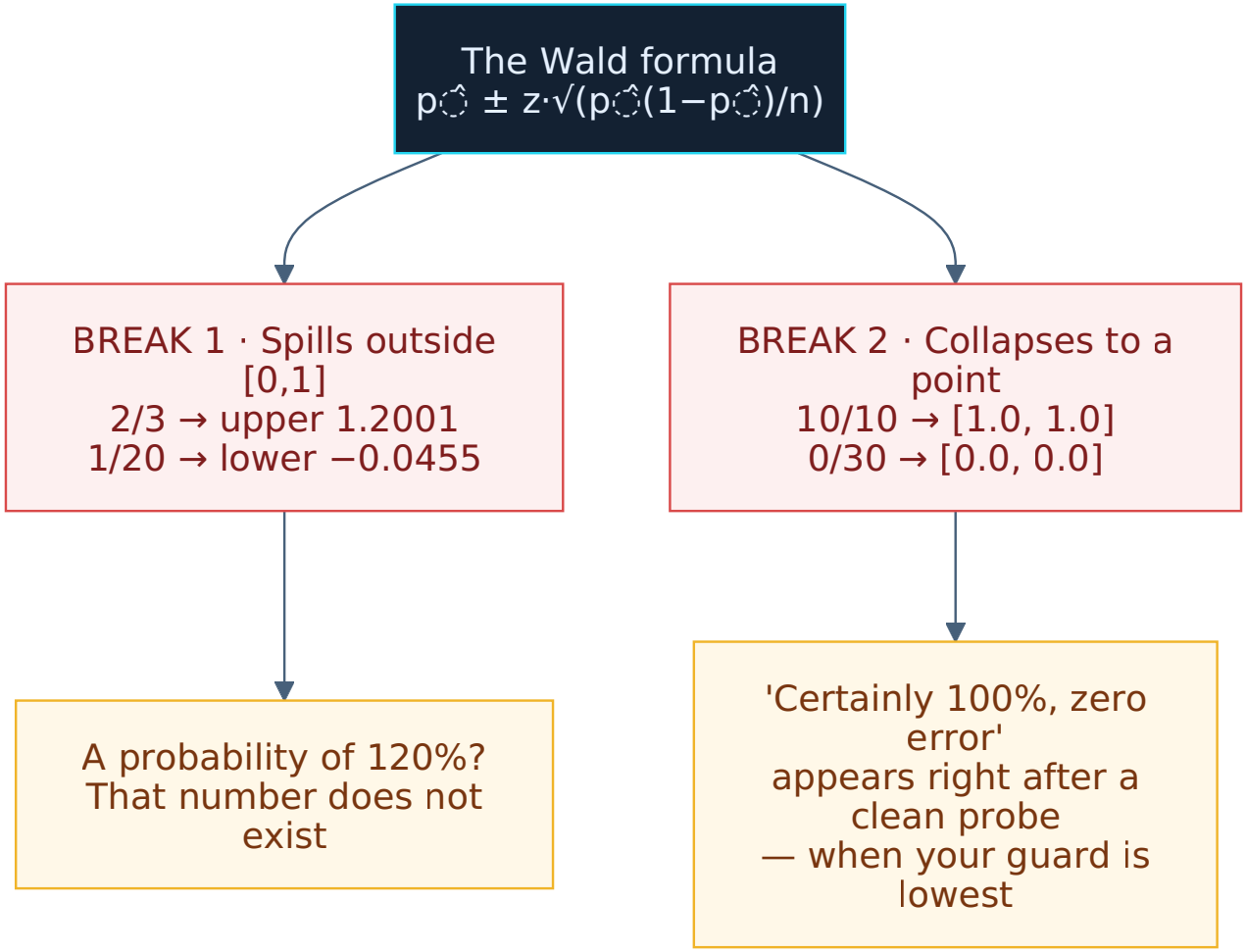
II.2 — Why not a simpler formula

There is an older, simpler confidence interval formula called **Wald**:

```
p̂ ± z·√( p̂(1−p̂)/n )
```

It breaks exactly where you need it most:

Observation	Wald 95%	Wilson 95%
8/10	[0.5521, **1.0479**]	[0.4902, 0.9433]
2/3	[0.1332, **1.2001**]	[0.2077, 0.9385]
1/20	[**−0.0455**, 0.1455]	[0.0089, 0.2361]
10/10	[**1.0000, 1.0000**]	[0.7225, 1.0000]
0/30	[**0.0000, 0.0000**]	[0.0000, 0.1135]



Wilson never spills past the boundary and never collapses to zero width. For 10/10 it says `[0.7225, 1.0000]` — "*could be only 72%; you lack the data to rule that out*".

Intuition: what keeps Wilson from collapsing

Wilson is equivalent to **quietly adding about 2 phantom trials** to your data: one success, one failure.

The consequence: the interval's centre is **pulled toward 0.5**, and pulled hard when n is small.

```
 10/10  → p̂ = 1.00  but centre = 0.8612
100/100 → p̂ = 1.00  but centre = 0.9815
```

This is a **feature, not a bug**. It means Wilson **refuses to believe anything absolutely** based on a handful of samples. To make it trust you at 99%, you must hand it a great deal of evidence.

At large n those two phantom trials vanish and Wilson converges to Wald. The difference between the formulas **exists only in the small-data regime** — exactly where probe-first lives, exactly where E14 and E23 operate.

II.3 — The formula, parameter by parameter

Four input parameters

Symbol	Meaning	Example
`n`	total trials	10
`k`	successes	8
`p̂`	`k / n` — observed rate	0.8
`z`	threshold for the confidence level	1.959964

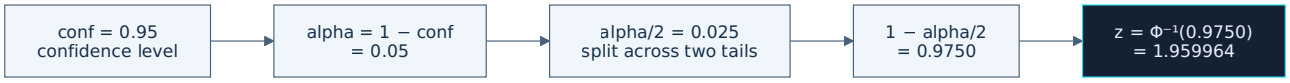
The first three come from data. The fourth is your choice.

The z table — computed, not looked up

`z` (the z-score) answers "*how certain do you want to be*". More certainty → larger z → wider interval.

```
conf = 0.90   alpha = 0.10   1 − alpha/2 = 0.9500   z = 1.644854
conf = 0.95   alpha = 0.05   1 − alpha/2 = 0.9750   z = 1.959964
conf = 0.99   alpha = 0.01   1 − alpha/2 = 0.9950   z = 2.575829
```

How those numbers arise:



Φ^{-1} is the inverse of the normal distribution. In Python: `NormalDist().inv_cdf(...)`.

> **A commonly misremembered value.** The **two-tailed** z for 99% is `2.575829`. The number `2.33` is also often cited, but that is the **one-tailed** z for 99% — using it by mistake yields an interval narrower than reality. This is precisely why this document runs the machine instead of consulting memory.

The Wilson formula

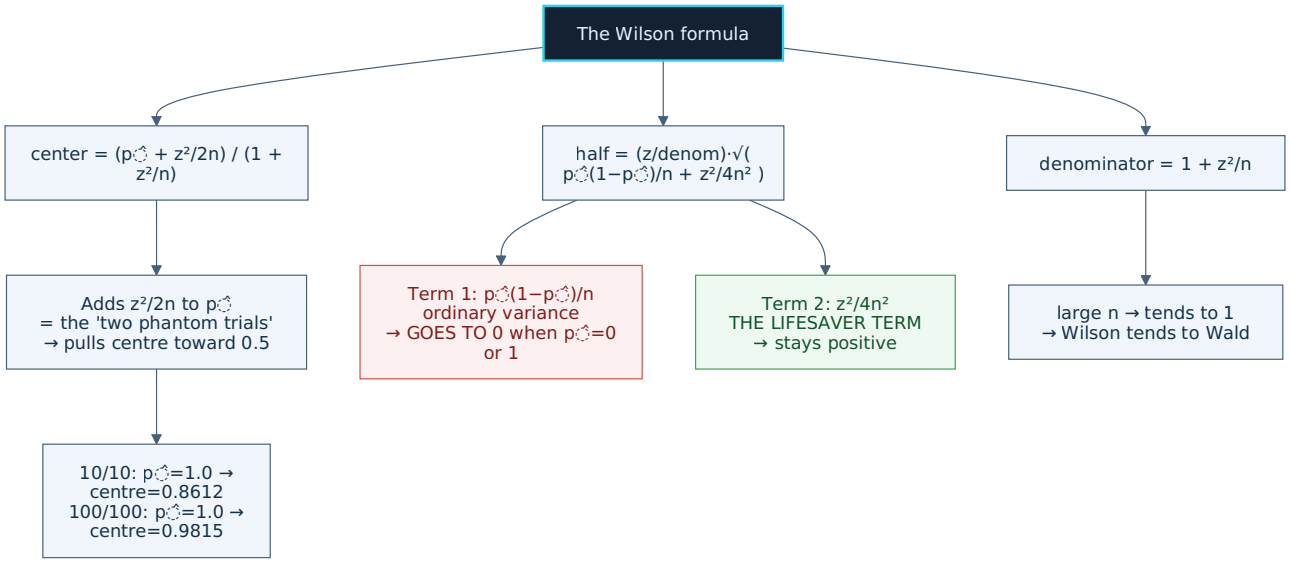
```
denominator = 1 + z²/n

center      = ( p̂ + z²/(2n) ) / denominator

half_width  = ( z / denominator ) · √( p̂(1−p̂)/n + z²/(4n²) )

CI = [ center − half_width , center + half_width ]
```

What each piece does



`center` — the interval's centre, and it does NOT equal p̂

Look at the numerator: `p̂ + z²/(2n)`. It **adds** `z²/(2n)` to the observed rate, then divides by `1 + z²/n`. At 95%, `z² = 3.841459`, so it adds roughly `1.92/n`.

This is the "two phantom trials" from II.2, written as a formula.

`half_width` — and the lifesaver term

Inside the square root there are **two** terms:

- `p̂(1−p̂)/n` — ordinary variance. Small when p̂ is near 0 or 1; small when n is large.
- `z²/(4n²)` — **the lifesaver term**. When p̂ = 0 or p̂ = 1 the first term goes to exactly 0 and Wald collapses. This one stays positive, so Wilson retains width.

Verified with real numbers, k=10, n=10, conf=95%:

```
p̂(1−p̂)/n = 0.000000     ← first term dies outright
z²/(4n²)  = 0.009604      ← this term keeps the interval alive
√(sum)    = 0.098000
half      = 0.138766      ← nonzero
```

That is the entire difference between "a meaningful interval" and "collapse to a point".

`denominator = 1 + z²/n` — normalisation

Large n → `z²/n` goes to 0 → the denominator tends to 1 → Wilson tends to Wald. The difference **exists only in the small-data regime**.

II.4 — Full worked example

k = 8, n = 10, conf = 95%:

```
z          = 1.959964
z²         = 3.841459
p̂          = 0.8

z²/n           = 0.384146
denominator    = 1 + 0.384146 = 1.384146

p̂ + z²/(2n)    = 0.8 + 0.192073 = 0.992073
center         = 0.992073 / 1.384146 = 0.716740

p̂(1−p̂)/n      = 0.016000
z²/(4n²)       = 0.009604
sum            = 0.025604
√sum           = 0.160011
half           = (1.959964 / 1.384146) × 0.160011 = 0.226578

CI = [0.716740 − 0.226578, 0.716740 + 0.226578]
   = [0.490162, 0.943318]
```

Against intuition: p̂ = 0.80, but the interval runs from **0.49 to 0.94**. You cannot distinguish "coin flip" from "near perfect".

II.5 — The full lookup table

```
k/n      p̂        Wilson 90%           Wilson 95%           Wilson 99%
-------------------------------------------------------------------------------
3/3     1.000   [0.5258, 1.0000]     [0.4385, 1.0000]     [0.3114, 1.0000]
5/5     1.000   [0.6489, 1.0000]     [0.5655, 1.0000]     [0.4297, 1.0000]
10/10   1.000   [0.7871, 1.0000]     [0.7225, 1.0000]     [0.6011, 1.0000]
30/30   1.000   [0.9173, 1.0000]     [0.8865, 1.0000]     [0.8189, 1.0000]
100/100 1.000   [0.9737, 1.0000]     [0.9630, 1.0000]     [0.9378, 1.0000]

2/3     0.667   [0.2535, 0.9217]     [0.2077, 0.9385]     [0.1442, 0.9596]
6/10    0.600   [0.3516, 0.8058]     [0.3127, 0.8318]     [0.2482, 0.8721]
7/10    0.700   [0.4417, 0.8731]     [0.3968, 0.8922]     [0.3200, 0.9204]
8/10    0.800   [0.5408, 0.9314]     [0.4902, 0.9433]     [0.4008, 0.9599]
9/10    0.900   [0.6523, 0.9774]     [0.5958, 0.9821]     [0.4928, 0.9881]

19/20   0.950   [0.8040, 0.9888]     [0.7639, 0.9911]     [0.6817, 0.9941]
48/50   0.960   [0.8861, 0.9867]     [0.8654, 0.9890]     [0.8201, 0.9921]
70/100  0.700   [0.6202, 0.7693]     [0.6042, 0.7811]     [0.5726, 0.8025]
95/100  0.950   [0.9008, 0.9755]     [0.8882, 0.9785]     [0.8608, 0.9832]

1/20    0.050   [0.0112, 0.1960]     [0.0089, 0.2361]     [0.0059, 0.3183]
0/30    0.000   [0.0000, 0.0827]     [0.0000, 0.1135]     [0.0000, 0.1811]
```

Three readings:

(a) 3/3 versus 30/30. Both are "100% pass". But 3/3 guarantees only ≥ 43.9%, while 30/30 guarantees ≥ 88.7%. Same report line, vastly different information value.

This answers pain point **I.5.2** — the `[OBSERVED]` label cannot distinguish these.

(b) Changing the confidence level matters far less than changing n.

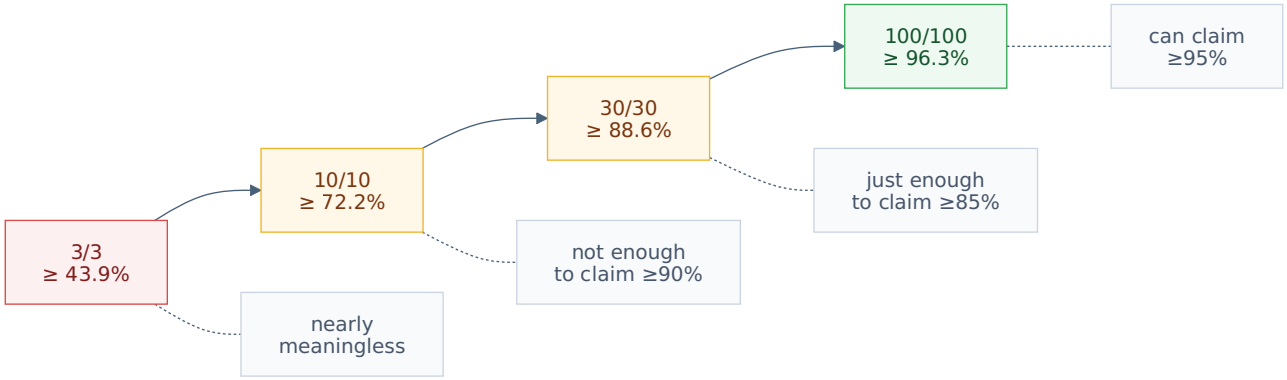
```
8/10 at 90% → lower bound 0.5408
8/10 at 99% → lower bound 0.4008        difference 0.14

p̂=0.8, n=10  → lower bound 0.4902
p̂=0.8, n=100 → lower bound 0.7112      difference 0.22
```

Do not agonise over 90 versus 95 — run more samples. That is the much larger lever.

(c) 0/30 → upper bound 0.1135. Running 30 times with zero errors does **not** mean the error rate is 0. It means the error rate **could be as high as 11.35%**.

Four numbers worth memorising



II.6 — How many samples is enough

This answers pain point **I.4.3** — E14 asked "is 16/20 enough" with no table to consult.

If **every** probe passes (k = n), the minimum samples for the 95% lower bound to reach threshold T:

```
T = 0.70  →  n =   9    (lower bound = 0.7009)
T = 0.80  →  n =  16    (lower bound = 0.8064)
T = 0.90  →  n =  35    (lower bound = 0.9011)
T = 0.95  →  n =  73    (lower bound = 0.9500)
T = 0.99  →  n = 381    (lower bound = 0.9900)
```

Claiming "≥90%" from a clean probe requires **35 consecutive passes**, not 10.

And if there **are** failures, to keep the lower bound ≥ 0.90:

```
n =  30  →  unreachable, even at 30/30
n =  50  →  at most  0 failures  (k=50,  lower = 0.9287)
n = 100  →  at most  4 failures  (k=96,  lower = 0.9016)
n = 200  →  at most 11 failures  (k=189, lower = 0.9042)
n = 500  →  at most 36 failures  (k=464, lower = 0.9019)
```

Note the first line: a 30-sample golden set can **never** demonstrate "≥90%", not even at 30/30. If your committed threshold is 90%, a 30-sample test set is the wrong-sized instrument — stop arguing about it and add samples.

Applied to E14

E14 measured Haiku's orientation guessing: **16/20** on the synthetically rotated set.

```
16/20  →  Wilson 95% = [0.5895, 0.9208]
```

From 59% to 92%. Consulting the table: claiming "≥80%" requires 16 samples **all passing**; here there are 4 failures, so it needs considerably more. Claiming "≥90%" needs at least 35 clean samples, or roughly 100 samples with 4 failures.

Practical conclusion: **20 samples is too few to decide**, and the table says how many more are needed.

II.7 — The zero-failure case

When `k = 0` the question inverts: *what is the maximum possible error rate?*

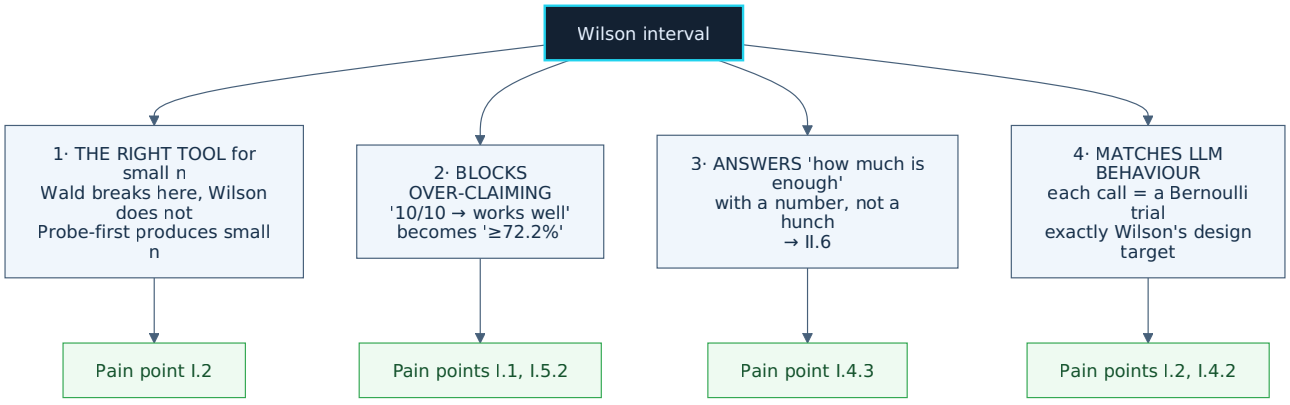
```
0/10  → upper bound = 0.2775      (rule of three: 3/n = 0.3000)
0/30  → upper bound = 0.1135      (3/n = 0.1000)
0/100 → upper bound = 0.0370      (3/n = 0.0300)
0/300 → upper bound = 0.0126      (3/n = 0.0100)
```

The **rule of three** is a mental shortcut: zero failures in n trials means the error rate could be around `3/n`. It approximates Wilson well (Wilson is slightly more conservative). Use it for mental arithmetic; use Wilson in the report.

This is also the correct reading of E14's result: "*never reported a false positive across 100 pages*" means `0/100 → error rate could reach 3.70%`, not "error rate is zero".

II.8 — Why Wilson fits probe-first

Summarising Part II with four reasons, ordered by importance:



But Part II has resolved only **two** of the six pain points. Remaining:

Pain point	Unresolved	Addressed in
I.4.1 — 100 pages / 6 case files	Non-independent samples	**III.2**
I.4.2 — judge J = 0.465	The ruler is bent too	**III.3**
I.5.1 — `rel_band = 0.05`	Noise band from a constant	**III.4**
—	Plus one thing Part II has not mentioned: Wilson **does not actually achieve 95%**	**III.1**

PART III

DEPTH

The four remaining pain points, plus one uncomfortable truth about Wilson itself. This is the longest part — and III.5 is the most important section in the document.

III.1 — The truth about "95%"

III.1.1. An uncomfortable question

Wilson claims to be a "95% interval". The question: **does it actually achieve 95%?**

The answer: **no, and it misses fairly often.**

This is not nitpicking. If the harness intends to use this number as a gate, it must know that the label says 95 while the delivery is something else.

III.1.2. What coverage means

Analogy: you set a trap and say "*this trap catches 95% of the mice that pass*". **Coverage** is going out and counting how many it actually catches. If the real figure is 91%, the "95%" label is false advertising.

For a confidence interval: you say "95% interval", meaning that if you repeated the experiment endlessly, 95% of the intervals produced would contain the true value. **Coverage** is the rate actually achieved.

III.1.3. Measured by enumerating every possibility

Toy problem: n = 10 trials, the truth is **p = 0.30** (we play God and know it in advance).

Enumerate **every** possible outcome — only 11 cases, k from 0 to 10:

```
 k   probability        Wilson interval     contains 0.30?
 0       0.028248      [0.0000, 0.2775]        NO   ←
 1       0.121061      [0.0179, 0.4042]        yes
 2       0.233474      [0.0567, 0.5098]        yes
 3       0.266828      [0.1078, 0.6032]        yes
 4       0.200121      [0.1682, 0.6873]        yes
 5       0.102919      [0.2366, 0.7634]        yes
 6       0.036757      [0.3127, 0.8318]        NO   ←
 7       0.009002      [0.3968, 0.8922]        NO   ←
 8       0.001447      [0.4902, 0.9433]        NO   ←
 9       0.000138      [0.5958, 0.9821]        NO   ←
10       0.000006      [0.7225, 1.0000]        NO   ←

Sum of probability over "yes" rows = 0.924403
```

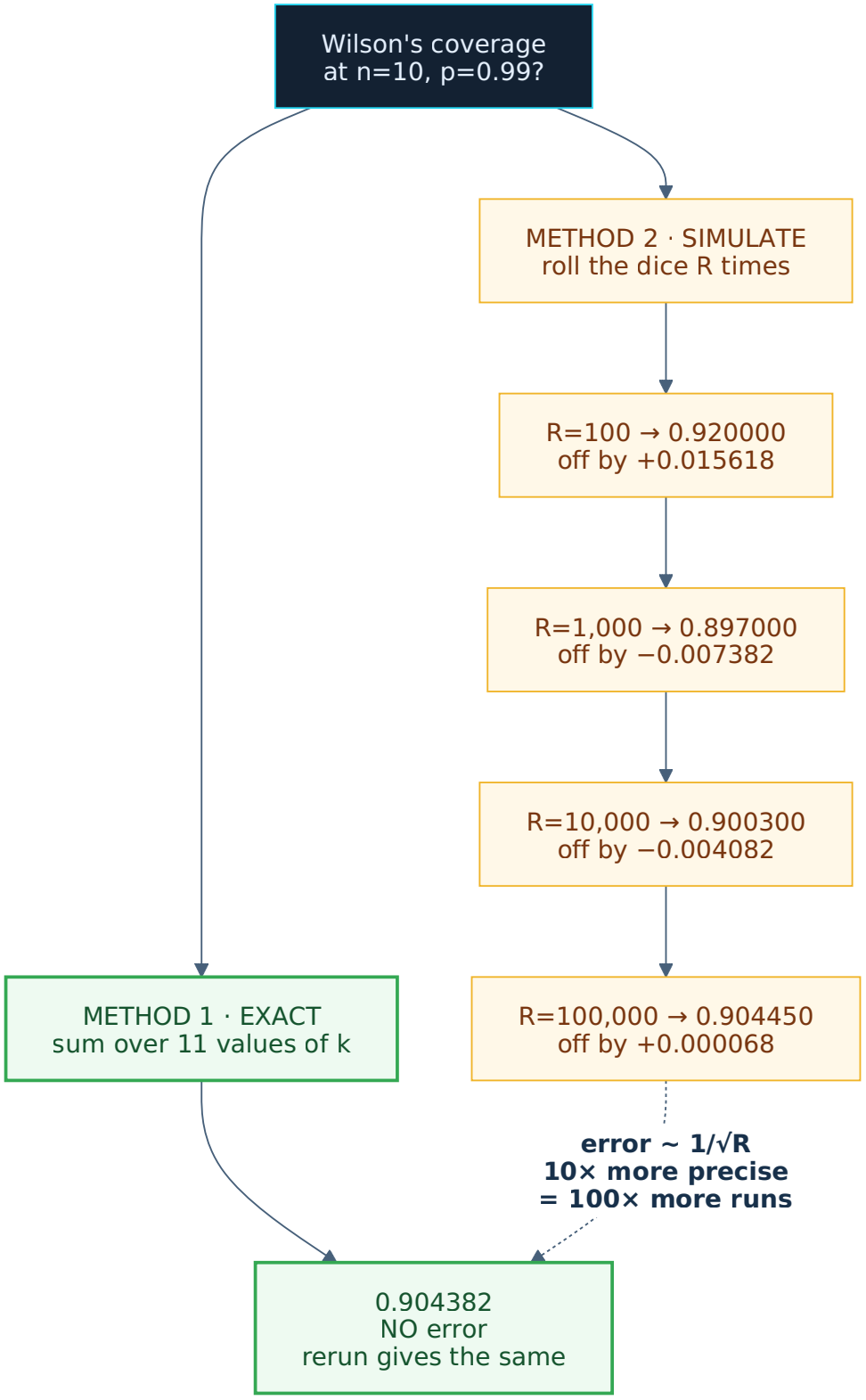
Wilson **promises** 0.95. Reality is **0.9244**. Short by 0.0256.

Read plainly: run this experiment 1000 times and roughly **76 of them** produce a Wilson interval that does **not** contain the truth — while the label says only 50.

III.1.4. Aside: Monte Carlo error

There is another way to answer that question: **simulation**. Instead of enumerating, roll the dice many times and count. That approach is called **Monte Carlo**.

Because it is random, the result differs slightly on each run — that difference is **Monte Carlo error**.



Why this matters here. When this document says "computed exactly", it means the number is not an estimate — rerun it 1000 times and it is identical. Conversely, several tables in III.2 and III.3 **do** use simulation; those all state a `seed` for reproducibility.

This is how the document labels the hardness of its own numbers — in the same spirit as the harness's `[OBSERVED]` / `[ASSUMED]` distinction.

III.1.5. Why larger n does not fix it

The cause: `k` **is an integer.** It jumps 0, 1, 2, 3… rather than sliding smoothly. Each jump moves coverage in a lump, and that lump almost never lands exactly on 0.95.

n	p=0.1	p=0.3	p=0.5	p=0.9
10	0.9298	0.9244	0.9785	0.9298
50	0.9703	0.9567	0.9351	0.9703
100	0.9364	0.9372	0.9431	0.9364
500	0.9563	0.9547	0.9456	0.9563
1000	0.9491	0.9509	0.9463	0.9491
5000	0.9496	0.9500	0.9507	0.9496

The oscillation **narrows** but **never settles at 0.95**. Even at n=5000 it still ripples.

This is not a Wilson defect. **Every** method suffers it, because it is inherent to count data.

III.1.6. The whole picture: sweeping the p grid

Sweeping p from 0.01 to 0.99, comparing three methods:

n	Wilson			Wilson-CC			Clopper-Pearson		
	mean	min	%p below .95	mean	min	%below	mean	min	%below
10	0.9553	0.9044	42.4%	0.9813	0.9599	0.0%	0.9835	0.9623	0.0%
20	0.9538	0.9245	40.4%	0.9774	0.9586	0.0%	0.9760	0.9586	0.0%
30	0.9524	0.9298	40.4%	0.9735	0.9572	0.0%	0.9735	0.9538	0.0%
50	0.9501	0.9106	43.4%	0.9693	0.9527	0.0%	0.9684	0.9534	0.0%
100	0.9492	0.9206	57.6%	0.9657	0.9513	0.0%	0.9649	0.9543	0.0%

Three points:

(a) Wilson is right on average, wrong locally. The mean sits near 0.95, so it looks fine in aggregate. But **40–58% of p values** yield coverage below 0.95, with the worst at **0.9044**.

(b) Larger n does not rescue it. At n=100 the miss rate is **higher** than at n=10 (57.6% vs 42.4%).

(c) The other two never miss — but their means rise to 0.966–0.983, i.e. **overly conservative**. You pay in wider-than-necessary intervals.

> **There is no perfect option.** Only: occasionally overconfident (Wilson), or always overconservative (the other two).

III.1.7. Continuity correction — the formula

The **continuity correction** (CC) is a corrected Wilson, proposed by Newcombe in 1998:

```
lower = [ 2np̂ + z² − 1 − z·√( z² − 2 − 1/n + 4p̂(n(1−p̂) + 1) ) ] / [ 2(n + z²) ]

upper = [ 2np̂ + z² + 1 + z·√( z² + 2 − 1/n + 4p̂(n(1−p̂) − 1) ) ] / [ 2(n + z²) ]

  k = 0  →  lower = exactly 0
  k = n  →  upper = exactly 1
```

Versus plain Wilson: the denominator changes from `n + z²` to `2(n + z²)`; the numerator gains `±1`; the radicand gains `∓2 − 1/n` and `±1` inside the bracket.

That `±1` is the "continuity" — it shifts the interval by **half a counting unit**, compensating for `k` moving in integer steps. Exactly the problem raised in III.1.5.

III.1.8. Clopper-Pearson — the third method

There is one more method, **Clopper-Pearson**, usually called "exact".

It uses no normal approximation, computing **directly from the binomial distribution**: find the smallest and largest p under which the observation remains "plausible". Because it is direct, its coverage is **never** below the nominal level.

The cost: usually wider, and more expensive to compute (requires bisection on the binomial).

III.1.9. Three-method comparison table

```
      k/n              Wilson           Wilson-CC     Clopper-Pearson     w(W)   w(CC)   w(CP)
    0/10   [0.0000,0.2775] [0.0000,0.3445] [0.0000,0.3085]   0.2775  0.3445  0.3085
    1/10   [0.0179,0.4042] [0.0052,0.4588] [0.0025,0.4450]   0.3863  0.4536  0.4425
    5/10   [0.2366,0.7634] [0.2014,0.7986] [0.1871,0.8129]   0.5268  0.5972  0.6258
    8/10   [0.4902,0.9433] [0.4422,0.9646] [0.4439,0.9748]   0.4532  0.5224  0.5309
    9/10   [0.5958,0.9821] [0.5412,0.9948] [0.5550,0.9975]   0.3863  0.4536  0.4425
   10/10   [0.7225,1.0000] [0.6555,1.0000] [0.6915,1.0000]   0.2775  0.3445  0.3085
    0/30   [0.0000,0.1135] [0.0000,0.1413] [0.0000,0.1157]   0.1135  0.1413  0.1157
   27/30   [0.7438,0.9654] [0.7232,0.9738] [0.7347,0.9789]   0.2216  0.2506  0.2442
   30/30   [0.8865,1.0000] [0.8587,1.0000] [0.8843,1.0000]   0.1135  0.1413  0.1157
   48/50   [0.8654,0.9890] [0.8514,0.9930] [0.8629,0.9951]   0.1236  0.1416  0.1323
  88/100   [0.8019,0.9300] [0.7960,0.9337] [0.7998,0.9364]   0.1281  0.1377  0.1367
  96/100   [0.9016,0.9843] [0.8949,0.9871] [0.9007,0.9890]   0.0827  0.0922  0.0883
 100/100   [0.9630,1.0000] [0.9539,1.0000] [0.9638,1.0000]   0.0370  0.0461  0.0362
    2/3    [0.2077,0.9385] [0.1253,0.9823] [0.0943,0.9916]   0.7308  0.8570  0.8973
    3/3    [0.4385,1.0000] [0.3100,1.0000] [0.2924,1.0000]   0.5615  0.6900  0.7076
    1/20   [0.0089,0.2361] [0.0026,0.2694] [0.0013,0.2487]   0.2272  0.2668  0.2475
```

Note the `100/100` row: Clopper-Pearson yields an interval **narrower than Wilson** (0.0362 vs 0.0370), despite never missing coverage. At the boundary `k = n`, CP beats both — contrary to its "overly conservative" reputation.

III.1.10. What CC costs

```
  k/n      Wilson lower → CC        loss      extra width
  8/10     0.4902 → 0.4422          0.0480     +15.3%
 10/10     0.7225 → 0.6555          0.0670     +24.1%
 27/30     0.7438 → 0.7232          0.0206     +13.1%
 30/30     0.8865 → 0.8587          0.0278     +24.5%
 88/100    0.8019 → 0.7960          0.0059      +7.5%
 96/100    0.9016 → 0.8949          0.0067     +11.5%
100/100    0.9630 → 0.9539          0.0091     +24.6%
460/500    0.8929 → 0.8918          0.0011      +4.1%
```

The steepest cost lands on the most common case: **all-pass** (10/10, 30/30, 100/100) each lose about 24%.

III.1.11. Does it change decisions? Almost never

This is III.1's most important finding.

Threshold `lower bound ≥ 0.90`, maximum failures allowed:

```
n= 30   Wilson=unreachable   CC=unreachable   CP=unreachable
n= 50   Wilson=0             CC=0             CP=0
n=100   Wilson=4             CC=3             CP=4      ← the ONLY difference
n=200   Wilson=11            CC=11            CP=11
n=500   Wilson=36            CC=36            CP=36
```

Across 5 configurations there is exactly 1 difference. The Wilson-vs-CC debate almost never **changes a gate decision**; it only changes the printed number.

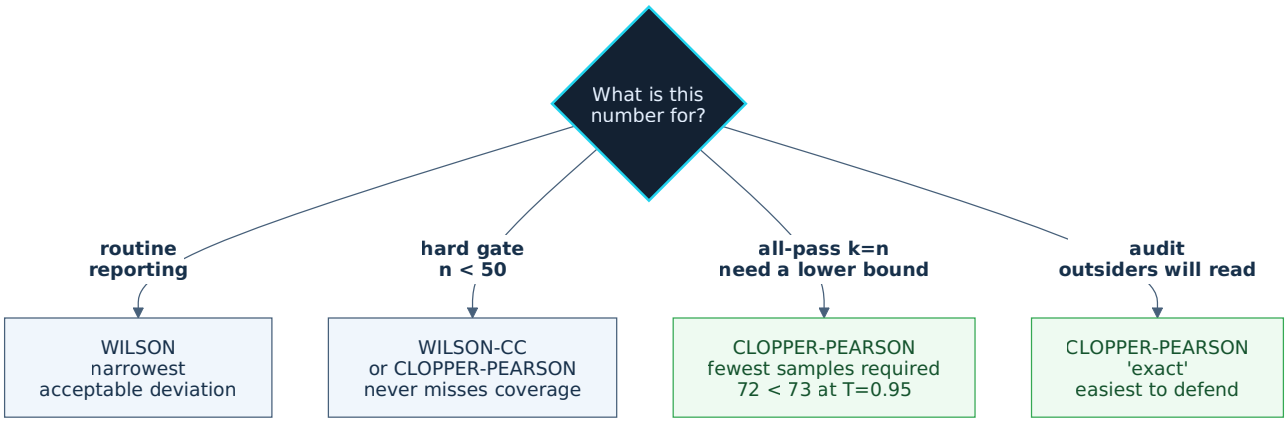
But minimum sample sizes differ substantially:

```
T=0.80:  Wilson= 16   CC= 21   CP= 17
T=0.90:  Wilson= 35   CC= 45   CP= 36
T=0.95:  Wilson= 73   CC= 92   CP= 72
T=0.99:  Wilson=381   CC=476   CP=368
```

CC demands roughly **29% more samples**. If the minimum-sample table (II.6) becomes a harness rule, choosing Wilson or CC is a real cost difference.

And the CP column is notable again: at T=0.95 and T=0.99 it demands **fewer samples than Wilson** (72 < 73, 368 < 381), despite coverage always ≥ 0.95.

III.1.12. Which to use



And a reading rule: when you write "Wilson95", read it as **"roughly 92–95%"**, not "exactly 95%".

III.2 — When samples are not independent

> **Resolves pain point I.4.1** — E23 ran 100 pages but from only 6 case files.

III.2.1. The silent assumption everyone violates

Every formula in Part II and III.1 rests on **one assumption**: trials are **independent** — this result says nothing about the next.

In both repos' reality, that assumption is violated almost by default.

Analogy: you poll 100 people, but they are all one family. You do not have 100 opinions — you have roughly 5 opinions, each repeated 20 times.

The consequence: Wilson yields an interval **narrower than the truth**. This errs in the dangerous direction — you are more confident than you are entitled to be.

III.2.2. First: the aggregate number also hides the break

A related problem, stated first because it is simpler.

A 100-image golden set split into 5 types × 20 (**stratified** = partitioned by data type):

```
type                 k/n     p̂       Wilson95              width
clear A4 paper      20/20   1.000   [0.8389, 1.0000]      0.1611
blurry paper        19/20   0.950   [0.7639, 0.9911]      0.2272
skewed photo        18/20   0.900   [0.6990, 0.9721]      0.2732
handwriting         17/20   0.850   [0.6396, 0.9476]      0.3081   ← could be just 64%
photocopy           18/20   0.900   [0.6990, 0.9721]      0.2732
-------------------------------------------------------------
TOTAL               92/100  0.920   [0.8500, 0.9589]      0.1089
```

The TOTAL row looks great: "≥85%, fine". But the **handwriting** stratum guarantees only **≥64%**.

Rule: golden sets must be stratified, with **a separate Wilson per stratum**. The number that drives decisions is the **worst stratum**, not the average.

The price: 20 samples per stratum means very wide intervals. That is not a defect of the method — it is the truth about your 100-image golden set.

(Stratification has a side effect many people miss: more strata means more false alarms. See III. 5.4.)

III.2.3. ICC — measuring the clumping

Back to the main issue. The quantity measuring how alike samples within a cluster are is the **ICC** (intra-cluster correlation).

Defined via variance:

```
ICC = between_cluster_variance / (between_cluster_variance + within_cluster_variance)
```

Analogy: you measure the height of 100 people across 5 classrooms. If the classrooms are equally tall on average, most variation is person-to-person → low ICC. If classroom A is all tall people and B all short, knowing the classroom nearly tells you the height → high ICC.

Plainly: **ICC = what fraction of the variation is due to "which cluster" rather than "which sample".** From 0 (fully independent) to 1 (knowing the cluster tells you the outcome).

III.2.4. The correction formula

```
deff  = 1 + (m − 1) · ICC          ← m = samples per cluster
n_eff = n / deff                   ← "effective" sample size
```

Then use `n_eff` in place of `n` in Wilson. `deff` is called the **design effect**.

With m = 20 (20 pages per case file), holding the pass rate at 90%:

```
ICC     deff    n_eff    Wilson95 (using n_eff)    width
0.0     1.00    100.0    [0.8256, 0.9448]          0.1191
0.1     2.90     34.5    [0.7704, 0.9695]          0.1991
0.3     6.70     14.9    [0.6212, 0.9626]          0.3415
0.5    10.50      9.5    [0.5958, 0.9821]          0.3863
0.8    16.20      6.2    [0.6097, 1.0000]          0.3903
```

Read the ICC=0.3 row: **100 pages are worth only 15 independent pages.**

III.2.5. ICC is computed from data, not declared

This is III.2's most important point, and the easiest to get wrong.

ICC is **not a parameter you guess**. There is a formula computing it **from the test results you already have** (an ANOVA-type estimator for binary data):

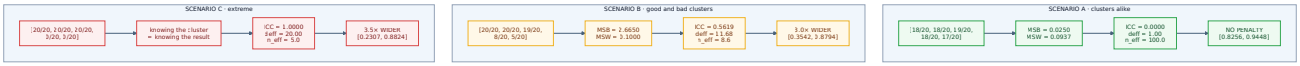
```
p̄    = total k / total n                          ← overall rate
MSB  = Σ nᵢ(p̂ᵢ − p̄)² / (K − 1)                    ← BETWEEN-cluster variation
MSW  = Σ nᵢ·p̂ᵢ(1 − p̂ᵢ) / (N − K)                 ← WITHIN-cluster variation
n₀   = (N − Σnᵢ²/N) / (K − 1)                     ← adjusted cluster size
ICC  = (MSB − MSW) / (MSB + (n₀ − 1)·MSW)         ← clamp to [0, 1]
```

(K = cluster count, N = total samples, n_i = size of cluster i, $\hat{p}_i$ = rate in cluster i)

If test results are stored per case file — which E23 **does** — then ICC is a **computation**, not a guess.

III.2.6. Three scenarios, real numbers

All three use 5 clusters × 20 samples.



Scenario A is notable: cluster correction is **not an automatic penalty**. If the clusters really are alike, you lose nothing. You are penalised only when the clusters differ — that is, only when there really is a problem.

III.2.7. The hard ceiling: n_eff never exceeds K/ICC

The most design-relevant finding here, and the one that applies directly to E23.

As `m → ∞` (scanning endless pages per case file):

```
n_eff = N / (1 + (m−1)·ICC)  →  K / ICC
```

```
ICC=0.1 → n_eff ceiling = 10.0 × cluster count
ICC=0.2 → n_eff ceiling =  5.0 × cluster count
ICC=0.3 → n_eff ceiling =  3.3 × cluster count
ICC=0.5 → n_eff ceiling =  2.0 × cluster count
ICC=0.8 → n_eff ceiling =  1.2 × cluster count
```

Applied to E23 — 6 case files:

ICC	n_eff ceiling
0.1	**60.0**
0.2	**30.0**
0.3	**20.0**
0.5	**12.0**

Scanning another 1000 pages from these 6 files **achieves nothing**. Progress requires **more case files**.

Verified in the other direction — holding N=100 fixed at ICC=0.2 and varying the clustering:

```
K=  2 clusters, 50 samples each  → deff=10.80  n_eff=  9.3   [0.5650, 0.9801]
K=  4 clusters, 25 samples each  → deff= 5.80  n_eff= 17.2   [0.7302, 0.9895]
K=  5 clusters, 20 samples each  → deff= 4.80  n_eff= 20.8   [0.7109, 0.9735]
K= 10 clusters, 10 samples each  → deff= 2.80  n_eff= 35.7   [0.7469, 0.9559]
K= 20 clusters,  5 samples each  → deff= 1.80  n_eff= 55.6   [0.7853, 0.9500]
K= 25 clusters,  4 samples each  → deff= 1.60  n_eff= 62.5   [0.8045, 0.9549]
K= 50 clusters,  2 samples each  → deff= 1.20  n_eff= 83.3   [0.8212, 0.9503]
K=100 clusters,  1 sample each   → deff= 1.00  n_eff=100.0   [0.8256, 0.9448]
```

Many small clusters beat few large ones. For the same 100 samples: 50 sources yield 83.3 effective samples; 2 sources yield only 9.3.

Golden set design rule: prioritise **source count**, not sample count. 2 pages from each of 50 case files beats 100 pages from 1 case file — by a factor of 9.

III.2.8. The trap: no default constant can be safe

The natural question: *if ICC cannot be computed yet, can we use a default, say 0.3?*

The answer: **no**. Checked (N=100, K=5, 90% pass):

```
true ICC    true lower bound    lower bound assuming 0.3    verdict
  0.05         0.7902                  0.6212               SAFE (more conservative)
  0.10         0.7704                  0.6212               SAFE
  0.20         0.7109                  0.6212               SAFE
  0.30         0.6212                  0.6212               exact
  0.50         0.5958                  0.6212               DANGEROUS (more optimistic)
  0.70         0.4869                  0.6212               DANGEROUS (off by 0.13)
```

Assuming 0.3 is safe only when the true ICC is **≤ 0.3**. When it exceeds — and scenario B in III. 2.6 gives ICC = **0.5619**, entirely realistic — it makes you **more confident than the truth**.

The general problem: "safe" depends on the very thing you do not know. **No default constant is safe.**

(This is also exactly the flaw in `rel_band = 0.05` in `hs:bakeoff` — pain point I.5.1. Same disease: a constant standing in for a computation.)

III.2.9. So what do you do with few clusters?

First, check whether an ICC estimate from few clusters is trustworthy at all.

Simulation (true ICC = 0.30, m = 20 per cluster, 300 replications, `seed=20260804`):

```
K clusters   true ICC   median estimate   5–95% range
   3           0.30         0.1396        [0.0000, 0.5789]
   5           0.30         0.1659        [0.0000, 0.6026]
  10           0.30         0.2133        [0.0113, 0.5688]
  20           0.30         0.2759        [0.0912, 0.4737]
  50           0.30         0.2884        [0.1570, 0.3951]
```

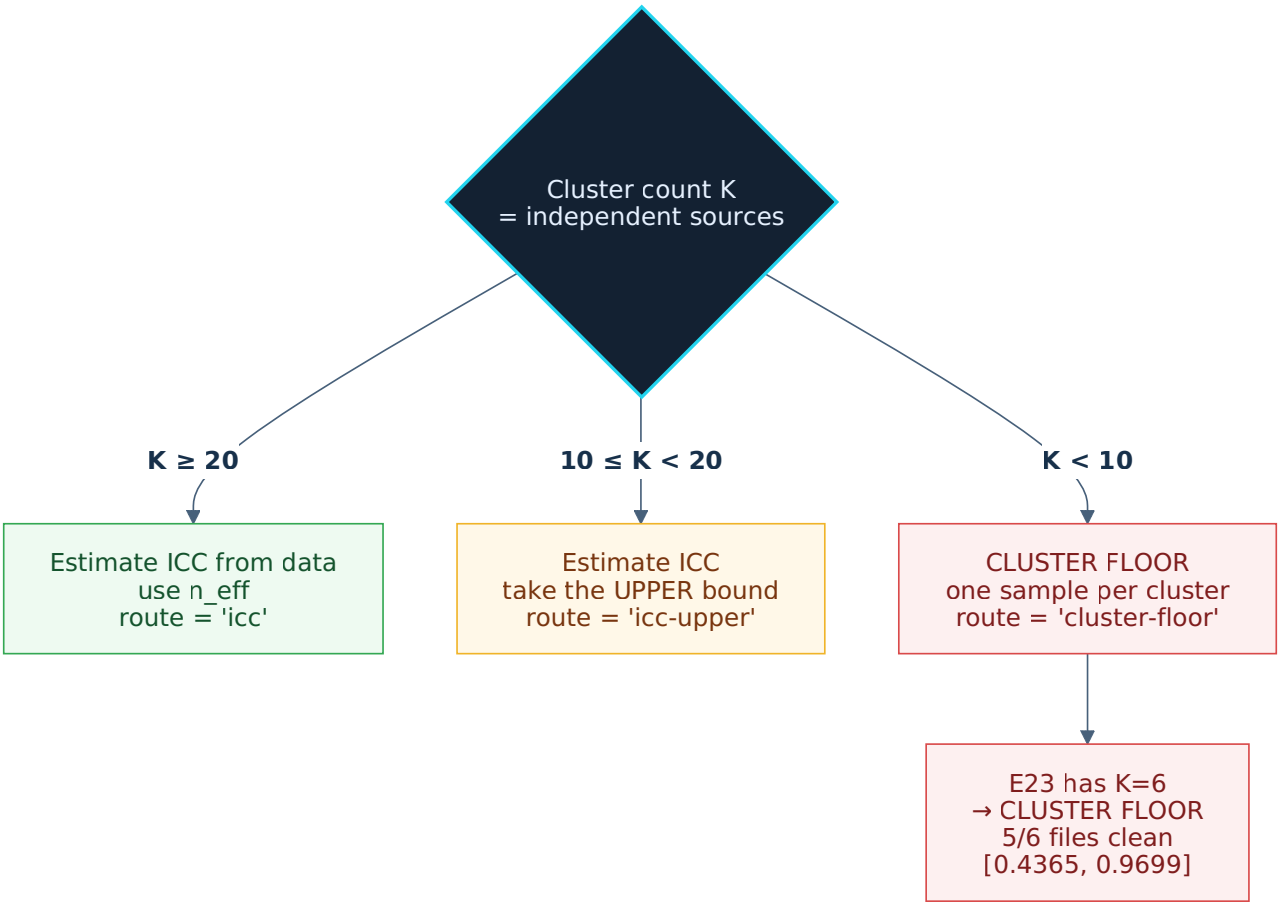
Below 10 clusters the ICC estimate is **systematically biased low** (0.14–0.17 when the truth is 0.30) and swings from 0 to 0.60. Useless — and worse: biased low means **penalising less than required**, again erring in the dangerous direction.

The remedy — the cluster-level floor: treat **each cluster as one sample**. A cluster passes if it meets the threshold, fails otherwise. Then Wilson on `clusters passed / K`.

```
scenario B:  3/5 clusters pass  →  [0.2307, 0.8824]
scenario C:  3/5 clusters pass  →  [0.2307, 0.8824]
```

Compared to the ICC path: scenario C (ICC = 1.0) yields **exactly the same result**. That is as it must be — the cluster floor *is* the ICC = 1 case, the worst possible.

III.2.10. The rule by cluster count



Cluster count K	Approach
K ≥ 20	Estimate ICC from data, use `n_eff`. The estimate is stable enough.
10 ≤ K < 20	Estimate ICC but take its **upper bound** (conservative).
K < 10	**Cluster floor** — one sample per cluster. Do not guess ICC.

This needs no invented constant. It is always determined by the data, and always errs conservatively.

(The cluster floor needs a "what counts as a passing cluster" threshold. Where that threshold comes from is an open question — see IV.10 Q6.)

III.3 — When AI does the scoring

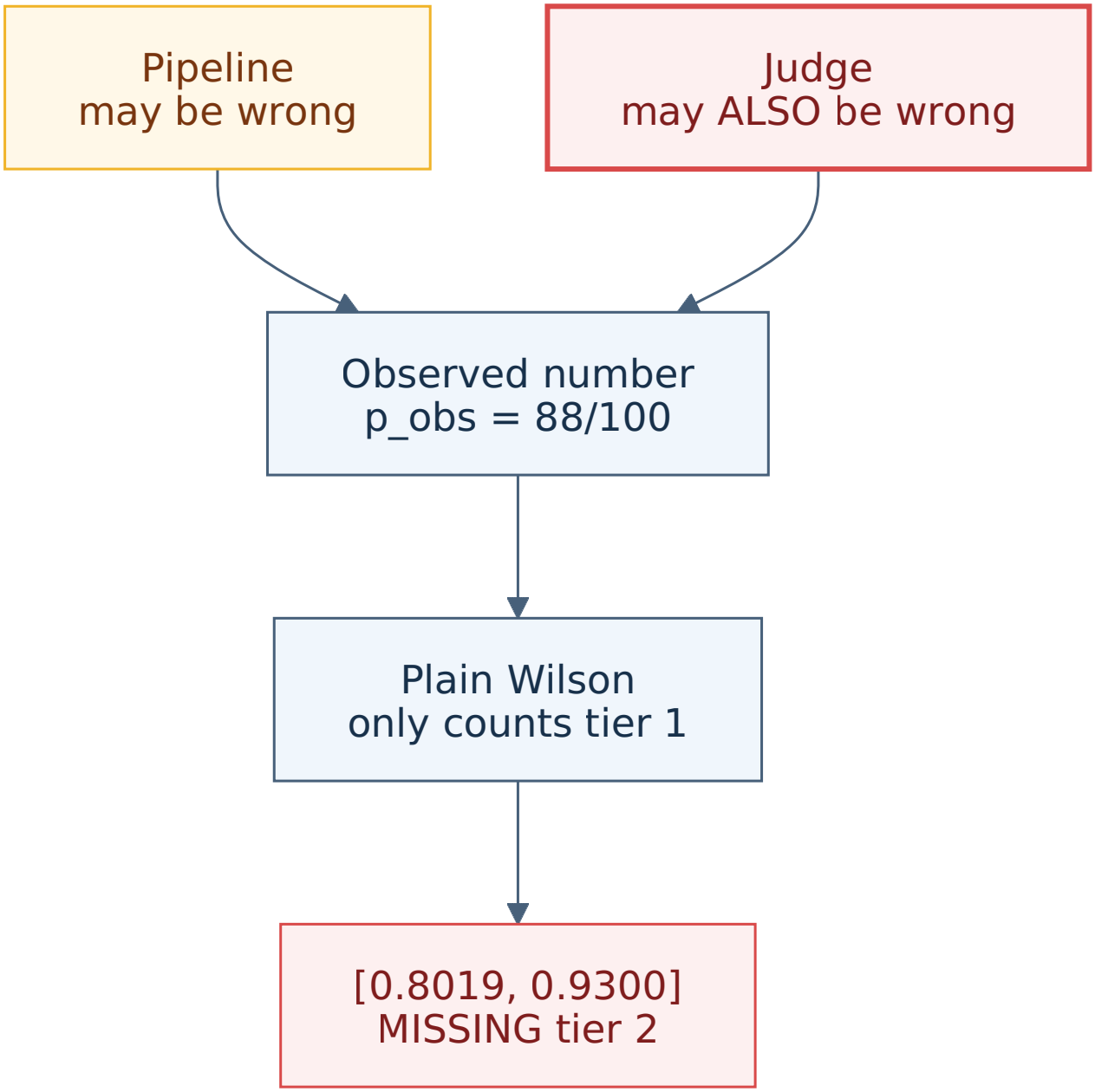
> **Resolves pain points I.4.2 and I.5.3** — E23's judge has J = 0.4650; the harness lets LLMs issue verdicts without anyone measuring the LLM.

III.3.1. The problem: the ruler is bent too

Analogy: you ask someone to proofread for spelling errors. They report "*88 of 100 sentences are correct*". But that person also misreads. So is 88 the real number, or the number seen through a warped lens?

E11 made the same point with a different image: "*measuring a table with a wooden ruler nobody has checked for straightness*".

Using an **LLM judge** gives you **two** error sources:



Everything in Part II and III.1–III.2 counts only tier 1. Tier 2 is ignored entirely.

This is the most important section for a harness that runs on LLMs.

III.3.2. A single accuracy number is not enough

The first instinct: "measure the judge's accuracy and factor it in". That instinct is **not enough**, because accuracy merges two entirely different failure modes.

The **confusion matrix** (a 2×2 table of judge verdict against truth):

```
                    truth: RIGHT     truth: WRONG
  judge says RIGHT         TP               FP
  judge says WRONG         FN               TN

  sens (sensitivity) = TP/(TP+FN)  = % of ACTUALLY-RIGHT cases caught correctly
  spec (specificity) = TN/(TN+FP)  = % of ACTUALLY-WRONG cases caught correctly
```

Three judges with near-identical accuracy but opposite behaviour:

```
LENIENT judge (passes freely)     acc=0.90   sens=0.9714   spec=0.7333
STRICT judge (fails freely)       acc=0.88   sens=0.8286   spec=1.0000
BALANCED judge                    acc=0.90   sens=0.9000   spec=0.9000
```

From the same observation `p_obs = 0.85`, the three yield entirely different conclusions:

```
LENIENT judge   →  p_true = 0.8277
STRICT judge    →  p_true = 1.0259   (exceeds 1 — a sign of model mismatch)
BALANCED judge  →  p_true = 0.9375
```

A gap of 0.11 between two judges with the same 0.90 accuracy.

Rule: the judge calibration record must store the **full confusion matrix** (TP/FN/TN/FP), not just accuracy.

The `p_true = 1.0259` case is also worth noting: exceeding [0,1] means the model is inconsistent with the data. It is a **warning signal**, not a result to silently clamp.

III.3.3. Rogan-Gladen — derived from scratch

Knowing how wrong the judge is lets you back out the true rate. The formula derives in 3 lines.

The judge says RIGHT in two cases: (actually right AND judge caught it) OR (actually wrong AND judge got it backwards):

```
p_obs = p_true·sens + (1 − p_true)·(1 − spec)
```

Expand and collect `p_true`:

```
p_obs = p_true·sens + 1 − spec − p_true + p_true·spec
p_obs − 1 + spec = p_true·(sens + spec − 1)
```

```
p_true = (p_obs + spec − 1) / (sens + spec − 1)        ← Rogan-Gladen
```

Round-trip verification to confirm no sign errors:

```
p_true=0.90 sens=0.95 spec=0.92 → p_obs=0.863000 → back to 0.900000  OK
p_true=0.75 sens=0.88 spec=0.80 → p_obs=0.710000 → back to 0.750000  OK
p_true=0.50 sens=0.99 spec=0.99 → p_obs=0.500000 → back to 0.500000  OK
p_true=0.30 sens=0.85 spec=0.90 → p_obs=0.325000 → back to 0.300000  OK
```

III.3.4. Judge bias distorts the number proportionally

The judge scores 100 new samples → 88 pass. What is the true rate?

```
perfect judge (100/100)          p_obs=0.88  →  p_true = 0.8800
judge 96% / 96%                  p_obs=0.88  →  p_true = 0.9130
judge 94% sens / 90% spec        p_obs=0.88  →  p_true = 0.9286
judge 90% / 90%                  p_obs=0.88  →  p_true = 0.9750
judge 85% / 85%                  p_obs=0.88  →  p_true = 1.0000
```

The last row: if the judge is only 85% accurate, the observed 88% is **consistent with a perfect 100% pipeline** — all 12 "failures" could be judge errors.

The reverse holds too, and is more dangerous: a lenient judge inflates `p_obs` artificially.

III.3.5. Youden's J — the denominator that governs everything

The denominator `sens + spec − 1` has a name: **Youden's J**. It measures how much better than guessing the judge is. J = 0 means useless (division by zero). J = 1 means perfect.

Because you divide by J, **every error is multiplied by 1/J**:

```
sens   spec      J      amplification factor 1/J
0.99   0.99   0.980            1.02
0.95   0.95   0.900            1.11
0.90   0.90   0.800            1.25
0.85   0.85   0.700            1.43
0.80   0.80   0.600            1.67
0.75   0.75   0.500            2.00   ← practical threshold
0.70   0.70   0.400            2.50
0.60   0.60   0.200            5.00
0.55   0.55   0.100           10.00
          ▲
```

Practical threshold: a judge below roughly 0.75/0.75 amplifies error by ≥2× — back-correction stops being meaningful. Such a judge should be **rejected**, not corrected.

III.3.6. Applied to E23 — a real judge, a real result

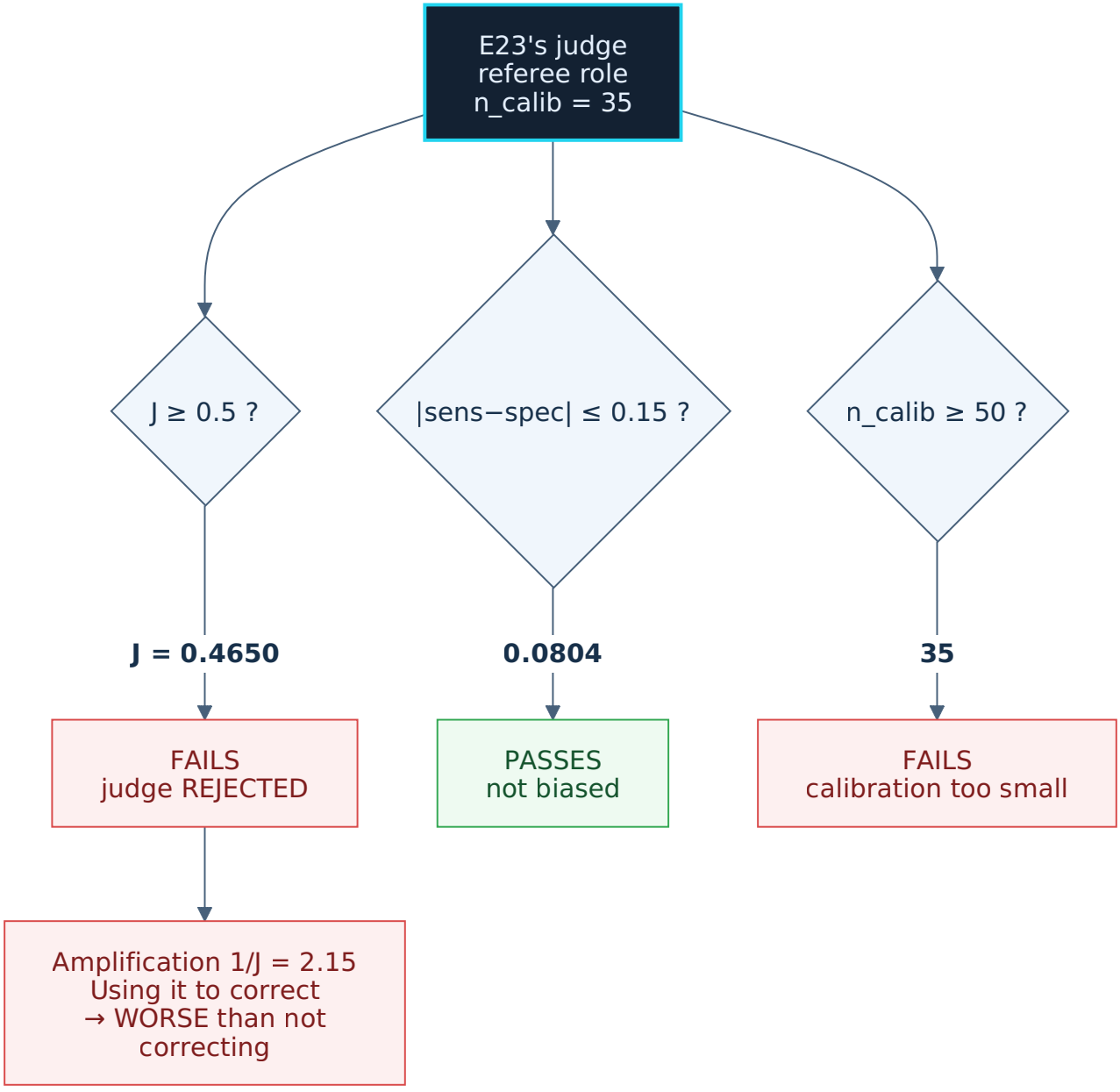
This is where the theory meets real data.

From table 4.2 of `E23-260803-llm-doc-truong-va-trong-tai.md`, reconstructing the referee LLM's confusion matrix:

```
                    software WRONG    software RIGHT
LLM says WRONG          TP = 9            FP = 5
LLM says RIGHT          FN = 4            TN = 17

n_calib = 35

sens = 9/13  = 0.6923      Wilson [0.4237, 0.8732]
spec = 17/22 = 0.7727      Wilson [0.5656, 0.8988]
Youden J     = 0.4650
|sens − spec| = 0.0804
```



Verified numerically. Using this judge to score 100 new pages yielding 88 passes:

```
naive (ignoring judge error):  [0.8019, 0.9300]   width 0.1281
combined with E23 judge error: [0.0000, 1.0000]   width 1.0000
```

The interval spans **all** of [0,1] — completely uninformative. That is the cost of J = 0.4650.

This does **not** make Role B useless. E23 concludes correctly that it is an **alarm bell** — it caught 9 of 13 real errors, including all 5 serial errors that previously required manual hunting, and two cases where the software fabricated a number that did not exist.

The meaning: **useful for summoning a human, not for replacing the human scorer.** That is exactly the two-tier boundary in III.3.11.

III.3.7. Combining both uncertainty tiers — and the real cost

Combining three uncertainty sources (p_obs, sens, spec) by splitting alpha three ways (**Bonferroni** style), then taking the outer envelope.

The pipeline scored 88/100 by the judge:

```
n_calib= 20 (19 right)  sens=[0.6176,0.9905]  spec=[0.5115,1.0000]  → p_true=[0.5826,1.0000]
w=0.4174
n_calib= 50 (48 right)  sens=[0.7789,0.9880]  spec=[0.7095,1.0000]  → p_true=[0.7033,1.0000]
w=0.2967
n_calib=100 (96 right)  sens=[0.8588,0.9883]  spec=[0.7820,0.9954]  → p_true=[0.7309,1.0000]
w=0.2691
n_calib=200 (193 right) sens=[0.9056,0.9872]  spec=[0.8570,0.9927]  → p_true=[0.7557,1.0000]
w=0.2443

CONTROL, naive (ignoring judge error):                                [0.8019,0.9300] w=0.1281
```

50 human-scored samples yield an interval 2.3× wider than naive.

Pushing further, with a modest target (combined interval only 50% wider than naive):

```
naive w=0.1281, target w ≤ 0.1922

n_calib=  100 → w=0.2549
n_calib=  200 → w=0.2368
n_calib=  400 → w=0.2251
n_calib=  800 → w=0.2169
n_calib= 1600 → w=0.2086
n_calib= 3200 → w=0.1969
n_calib= 6400 → w=0.1889   ← finally MEETS it
```

6400 human-scored samples. That is the real cost of full judge correction.

But there is a redeeming detail. If sens = spec = 0.96 were **known exactly** (infinite calibration):

```
p_true = [0.8281, 0.9674]   w = 0.1393   (naive = 0.1281)
→ only 1.09× wider than naive
```

The theoretical floor is 1.09×, but reality at n_calib=100 is 1.99×.

That gap comes almost entirely from (a) uncertainty in the calibration set itself and (b) Bonferroni's conservatism.

Meaning: **most of the cost is not the judge being bad — it is you not knowing how good the judge is.**

(Technical note: the combination here is a Bonferroni-style corner envelope — simple, always conservative, easy to audit. A tighter method (bootstrap or profile likelihood) would narrow it but is considerably more complex. See IV.10 Q5.)

III.3.8. Calibration set design: balanced, not random

Holding n_calib = 100 and varying the share of actually-wrong cases:

% wrong	npos/nneg	sens width	spec width	→ p_true width	
10%	90/10	0.1147	0.3643	0.3293	
20%	80/20	0.1278	0.2872	0.2939	
30%	70/30	0.1442	0.2434	0.2770	
50%	50/50	0.1573	0.1573	0.2549	← best
70%	30/70	0.2434	0.1442	0.2486	

spec is the bottleneck — it has only `nneg` samples to learn from.

The practical consequence: drawing the calibration set **randomly** from production where the pass rate is 95% gives roughly 5 wrong cases in 100 samples → spec becomes nearly unmeasurable.

Rule: the calibration set must be **deliberately balanced** (~50/50), not randomly drawn. That means **actively hunting for wrong cases** to include.

E14 already did this by instinct. The real dataset had no rotated pages, so they **manually rotated 20 pages** on a balanced schedule — exactly 5 pages per angle across {0°, 90°, 180°, 270°}, `seed=42`.

That is a deliberately balanced calibration set, built before this formula existed.

III.3.9. Correction TIGHTENS the gate, never loosens it

The safety question: can correction turn a failing gate into a passing one?

A genuinely good pipeline (p_true = 0.93), threshold `lower bound ≥ 0.85`:

```
judge 0.99: p_obs=0.9300 (k=93)  naive=PASS [0.8625,..]   combined=FAIL [0.8249,..]  ← flip
judge 0.96: p_obs=0.8956 (k=90)  naive=FAIL [0.8256,..]   combined=FAIL [0.7745,..]
judge 0.92: p_obs=0.8612 (k=86)  naive=FAIL [0.7786,..]   combined=FAIL [0.7126,..]
judge 0.88: p_obs=0.8268 (k=83)  naive=FAIL [0.7445,..]   combined=FAIL [0.6636,..]
judge 0.80: p_obs=0.7580 (k=76)  naive=FAIL [0.6677,..]   combined=FAIL [0.5252,..]
```

The only flip is **PASS → FAIL**, never the reverse. Correction only tightens — safe from a gate perspective.

But it also means it will **block genuinely good work** when the judge is poorly measured.

III.3.10. The visual trap: a bad judge looks like a narrow interval

Raw values, **before clamping to [0,1]**:

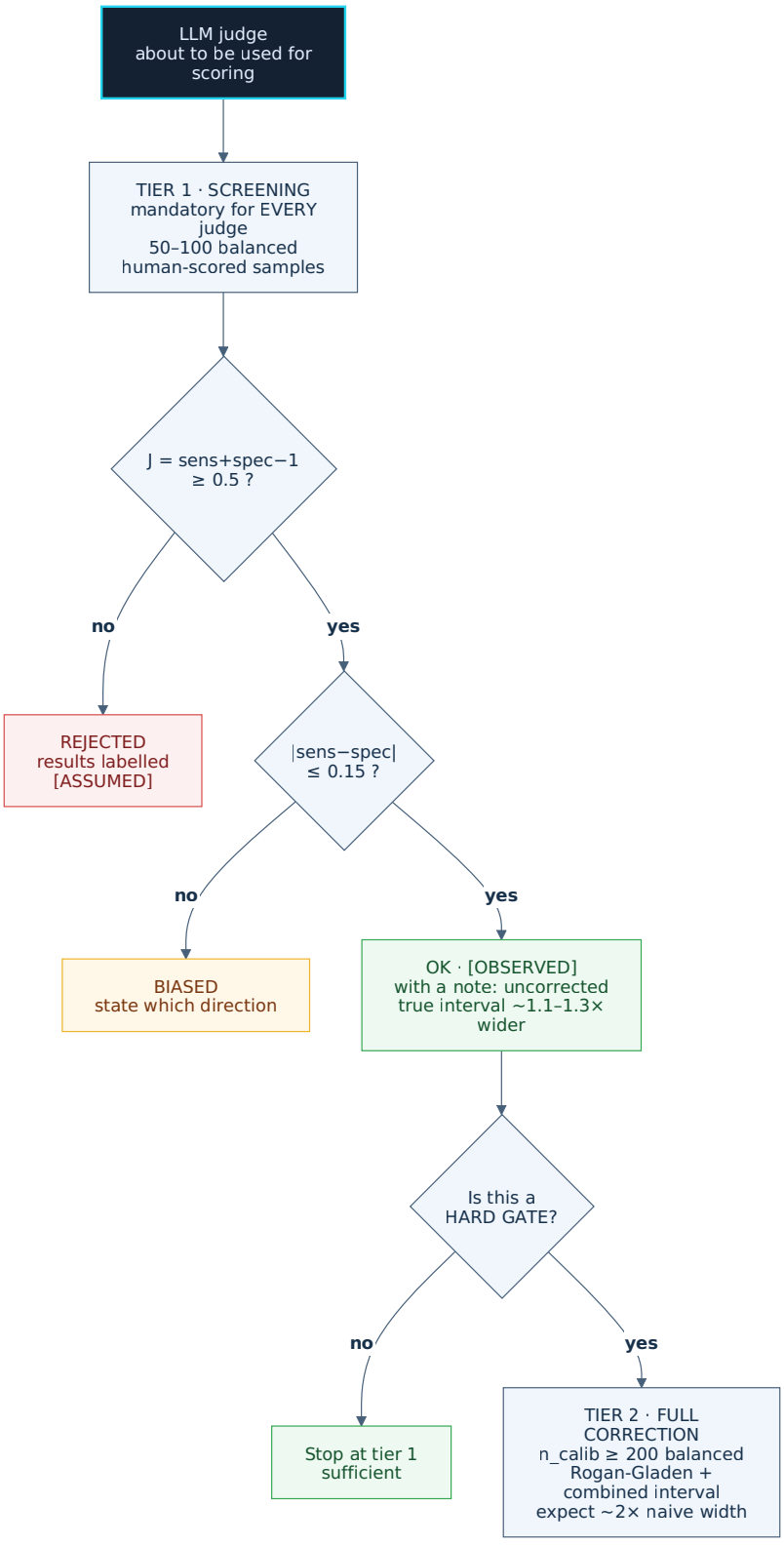
```
judge acc   raw (unclamped)              clamped           w after clamp
   0.99     [0.7559,  1.0512] w=0.2953   [0.7559,1.0000]     0.2441
   0.96     [0.7451,  1.1553] w=0.4102   [0.7451,1.0000]     0.2549
   0.92     [0.7440,  1.2827] w=0.5387   [0.7440,1.0000]     0.2560
   0.88     [0.7489,  1.4533] w=0.7044   [0.7489,1.0000]     0.2511
   0.80     [0.7769,  2.0953] w=1.3184   [0.7769,1.0000]     0.2231
   0.70     [0.8695,  7.5509] w=6.6814   [0.8695,1.0000]     0.1305   ← looks "narrowest"
```

The 70% judge has the **narrowest post-clamp interval in the table**. Reading only the last column, you would think it best. In truth its raw interval is **6.68** — utterly uninformative; it merely looks narrow because it burst through the ceiling and was cut off.

Rule: when the upper bound lands exactly on 1.0 (or the lower on 0.0) after clamping, the report must carry a **saturation flag**. An interval touching the boundary is not narrow — it has overflowed.

III.3.11. The two-tier rule

Full Rogan-Gladen correction is **too expensive for most cases** (6400 samples). Split it in two:



Tier 1 — cheap screening (every judge, mandatory): - Measure the judge on **50–100 balanced human-scored samples** (~50/50). - Report **sens and spec separately**, each with a Wilson interval, plus the full confusion matrix. - **Do not** correct. Use it purely as a quality gate: - `J < 0.5` → judge **rejected**, results labelled `[ASSUMED]`. - `|sens − spec| > 0.15` → judge **biased**, state the direction. - Passing both → results count as `[OBSERVED]`, with a note "*judge error uncorrected*".

Tier 2 — full correction (only when warranted): - Only for hard gates and irreversible decisions. - Requires n_calib ≥ 200, balanced. - Report the combined interval, with a saturation flag if it touches a boundary. - Expect ~2× the naive width. That is the price of honesty.

Tier 1 retains the **core value** (detecting biased judges, blocking useless ones) at a bearable cost.

III.3.12. Two judges do not help if they share a model

The money-saving idea: replace the human scorer with a second judge.

```
error correlation r=0.0:  P(both catch it) ≈ 0.8100  (= fully independent)
                   r=0.3:                  ≈ 0.8370
                   r=0.6:                  ≈ 0.8640
                   r=0.9:                  ≈ 0.8910
```

The higher r, the more the two judges behave as one. Two LLMs of the **same model and same prompt family** sit around r = 0.8–1.0 — essentially one judge run twice.

For two judges to add value they must be **different models with different lenses**, and even then r > 0.

> **E23 tried to do exactly this** — they planned "a small sample on a second engine" (Gemini Flash Lite) alongside Haiku. But `GEMINI_API_KEY` had been revoked by Google (HTTP 403, confirmed via two independent paths), so scope shrank to a single engine.
>
> They recorded that failure as a result rather than hiding it. But the consequence stands: E23's judge is currently a **single engine** with no cross-check.

III.3.13. Sequential verification multiplies

A related point that is easily forgotten.

A multi-step pipeline where each step has its own accuracy. Correct through the whole chain = the **product**:

```
step1   step2   both correct
0.95    0.90    0.8550
0.90    0.90    0.8100
0.99    0.95    0.9405
0.80    0.95    0.7600
```

Two "very good" steps (95% and 90%) compose to only **85.5%**.

With AI verification it is worse: if step 2 is an LLM checking step 1 and both rest on the same model, their errors are **correlated** — wherever the model is wrong, it cannot catch itself. The 0.8550 product is then **overly optimistic**.

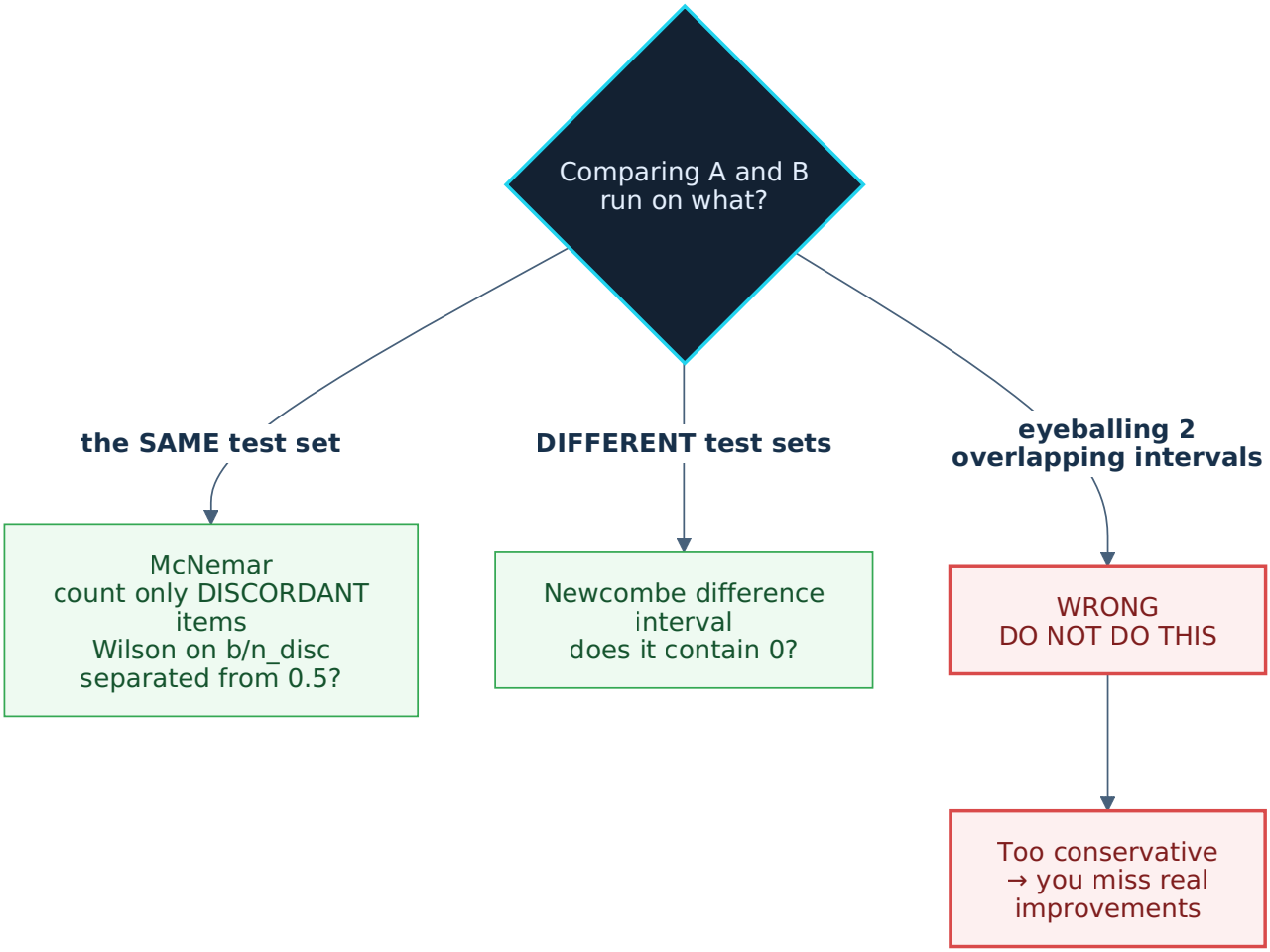
III.4 — Comparing two candidates

> **Resolves pain point I.5.1** — `hs:bakeoff` uses `rel_band = 0.05` as its noise band.

III.4.1. The question changes shape

So far the document answers "*how good is this pipeline?*". But the more common practical question is: **Is prompt A or prompt B better?**

That is a **different** question requiring a different method. Choosing the wrong tool is the most common error in this entire document.



III.4.2. The obvious mistake: comparing bare numbers

```
prompt A: 8/10 = 80%
prompt B: 6/10 = 60%
→ "pick A"
```

The real numbers:

```
A  8/10   p̂=0.800   Wilson95 = [0.4902, 0.9433]
B  6/10   p̂=0.600   Wilson95 = [0.3127, 0.8318]
```

The intervals overlap heavily. You have **no evidence** A beats B — you have noise.

With 100 samples:

```
A' 80/100  p̂=0.800   Wilson95 = [0.7112, 0.8666]
B' 60/100  p̂=0.600   Wilson95 = [0.5020, 0.6906]
```

Now they separate cleanly.

But comparing by "do the two intervals overlap" **is itself an error** — III.4.5.

III.4.3. Same test set → McNemar

Analogy: comparing two solutions by grading the same 100 students. You do not need the class averages — you only need the students where the two solutions **disagree**.

```
b       = items where A is right, B is wrong
c       = items where A is wrong, B is right
n_disc  = b + c
```

If the two are equally good, `b/n_disc` should sit near 0.5. Wilson on `b/n_disc`, then check whether it separates from 0.5.

Prompt A scores 80/100, prompt B scores 76/100, same test set:

```
b    c    n_disc   Wilson95(b/n_disc)     verdict
 6    2      8      [0.4093, 0.9285]      NOT enough evidence
10    6     16      [0.3864, 0.8152]      NOT enough evidence
12    8     20      [0.3866, 0.7812]      NOT enough evidence
 4    0      4      [0.5101, 1.0000]      DIFFERENT   ← only 4 items
 9    3     12      [0.4677, 0.9111]      NOT enough evidence
20   16     36      [0.3958, 0.7046]      NOT enough evidence
```

The `b=4, c=0` row: only **4 discordant items** and it already concludes, because the disagreement is entirely one-directional. Meanwhile `b=20, c=16` — 36 discordant items — cannot conclude, because it is nearly a tie.

> **The volume of disagreement matters far less than its lopsidedness.**

Rule: with the same test set, **always** use McNemar. Using two independent intervals discards most of the test set's power.

III.4.4. Different test sets → the difference interval

If A and B ran on different datasets, McNemar does not apply. But neither does eyeballing two intervals.

The dedicated formula for the **difference** (Newcombe):

```
lower_diff = (p̂₁ − p̂₂) − √( (p̂₁ − L₁)² + (U₂ − p̂₂)² )
upper_diff = (p̂₁ − p̂₂) + √( (U₁ − p̂₁)² + (p̂₂ − L₂)² )
```

(L, U are each group's own Wilson bounds)

Conclusion: if the difference interval **does not contain 0**, there is a difference.

```
      A          B     diff p̂    difference interval (Newcombe)   verdict
 80/100    76/100    +0.0400    [−0.0750, +0.1539]    no conclusion
 80/100    60/100    +0.2000    [+0.0731, +0.3185]    A beats B
  8/10      6/10     +0.2000    [−0.1870, +0.5211]    no conclusion
 90/100    80/100    +0.1000    [+0.0001, +0.1995]    A beats B
 45/50     40/50     +0.1000    [−0.0434, +0.2421]    no conclusion
190/200   175/200    +0.0750    [+0.0195, +0.1326]    A beats B
```

III.4.5. "The intervals overlap" is a reading error

This is the easiest thing to get wrong, and deserves its own section.

Take the `90/100 vs 80/100` row:

```
A = [0.8256, 0.9448]
B = [0.7112, 0.8666]
→ the intervals OVERLAP from 0.8256 to 0.8666
```

If you conclude "they overlap → no conclusion", you are **wrong**. The difference interval:

```
[+0.0001, +0.1995]   → does not contain 0 → CONCLUSIVE, A beats B
```

Why: each separate interval has already "paid" for its own uncertainty. Judging by overlap inadvertently double-counts that uncertainty, making the test far too conservative.

The practical consequence: you **miss real improvements**. A genuinely better prompt gets labelled "insufficient evidence" and goes unused.

III.4.6. Applied to `hs:bakeoff`

The current schema (`artifact-bakeoff-verdict.json`):

```
band    = max(rel_band × |rep(best)|, observed_spread_floor)
verdict = winner if gap > band, else tie_within_noise
rel_band default = 0.05
```

The design is already sound: it has a noise concept, an honesty floor (`observed_spread_floor`), and no silent truncation (`candidates: every candidate — losers included`).

The single flaw: **`rel_band = 0.05` is an invented constant** — the same disease as "default ICC 0.3" in III.2.8.

Replace with:

Situation	`tie_within_noise` condition
Same test set	Wilson on `b/n_disc` **contains 0.5**
Different test sets	Newcombe difference interval **contains 0**
All cases	**Forbid** "overlapping intervals" as a criterion

The advantage: the noise band is computed **from the data itself**, shrinks automatically as samples accumulate, and requires nobody to justify the number 0.05.

Plus one more constraint from III.5.2: bake-off must **report on a held-out set**, not the set used for selection.

III.5 — Process traps

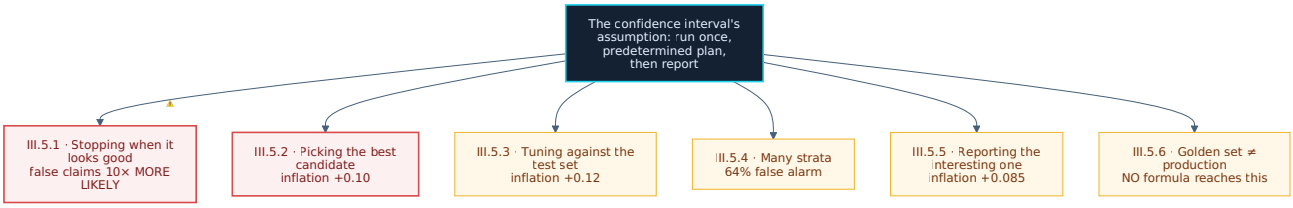
III.5.0. Why this section matters more than the mathematics

Everything from Part II through III.4 assumes one thing: you run the probe **once**, per a predetermined plan, then report the result.

Reality is otherwise. You add samples when you are unsure. You try several prompts and keep the best. You tune against the test set and then measure on that same set.

Each of those behaviours is **technically reasonable** but **violates the assumption** behind confidence intervals.

> **Comparing severity.** The continuity correction (III.1) changes numbers in the second decimal place and almost never changes a gate decision — 5 configurations, 1 difference.
>
> The traps below multiply the false-claim rate by **10×** and inflate scores by **0.10**.
>
> If you read only one section of this document, read III.5.



III.5.1. Stopping when it looks good (optional stopping)

The most dangerous trap for probe-first, because it describes exactly how people naturally work.

The scenario: add samples gradually, glance at Wilson occasionally, and when the lower bound clears the threshold, **stop and report**. It sounds entirely reasonable — it is precisely "probe until confident enough".

Simulation (`seed=99`): threshold `lower bound ≥ 0.70`, maximum 500 samples. Every claim is **false** because the truth sits below the threshold. False-claim rates:

true p	look/1	look/5	look/10	look/50	look/500
0.600	1.02%	1.13%	0.62%	0.02%	0.00%
0.650	4.10%	4.02%	2.55%	0.48%	0.00%
0.680	10.30%	8.62%	6.43%	2.18%	0.10%
0.695	16.67%	14.92%	11.72%	7.05%	1.15%
0.699	18.35%	17.48%	14.88%	8.82%	1.92%

Read the last row: the truth is 0.699 against a 0.70 threshold — **right at the line**. Looking every sample → **18.35%** chance of a false claim. Looking once at the end → **1.92%**. Nearly a **10×** difference.

Why: each look is another chance to catch a lucky moment. Look 500 times and it is nearly certain that at some point the data looks better than the truth — and you stop right there.

Note the other direction too: at `p=0.600` (far from the threshold) peeking is nearly harmless. **The danger scales with how close you are to the threshold** — which is exactly when you most want to look.

Rule: fix `n` **before running**, run it all, look **once**.

III.5.2. The winner's curse

Pick the best prompt among N candidates, then report its score. **That score is always inflated.**

Simulation (`seed=7`) — **all** candidates have the **same true ability = 0.80**:

```
candidates N   n_test   average inflation of the best score
           1       30              +0.0008
           2       30              +0.0391
           3       30              +0.0613
           5       30              +0.0807
          10       30              +0.1038
          20       30              +0.1257
          50       30              +0.1461
```

Picking 1 of 10 prompts → the reported score exceeds the truth by **0.10**. Not because that prompt is better — it is **not** — but because you selected the luckiest one.

Larger n_test helps but does not eliminate it:

```
N=10, n_test= 10 → +0.1661
N=10, n_test= 30 → +0.1054
N=10, n_test=100 → +0.0596
N=10, n_test=300 → +0.0349
```

Wilson computed on that same test set does NOT catch this bias.

Rule: select on set A, **report on set B** (held-out), never used for selection.

> `hs:bakeoff` **is directly exposed.** The essence of a bake-off is picking the best of several candidates. If the reported score comes from the very set used to select, it is inflated per the table above.
>
> This is the second constraint to add to `hs:bakeoff`, alongside replacing `rel_band` in III. 4.6.

III.5.3. Golden set leakage over time

The same family as III.5.2 but **far more insidious**, because it unfolds over weeks rather than within one sitting.

You use the golden set to tune prompts. Each tuning round is one round of **learning** that test set.

Simulation (`seed=11` , 50 samples, 5 variants per round, keeping the best):

```
tuning rounds   average inflation of the final score
            0            +0.0000
            1            +0.0631
            2            +0.0827
            5            +0.1033
           10            +0.1169
           20            +0.1284
```

After 10 rounds, inflation reaches **~0.12**. The golden set is **no longer an independent measure** — it has become part of the model.

Rule: golden sets have an **expiry**. After N tuning rounds you must refresh the samples, or keep a separate "sealed" set opened only at ship time.

> **A concrete risk for the OCR repo.** The E11 ground truth table is an expensive asset — 100 pages, human eyes on each, redone once after a misalignment. But it has served as the ruler for E14, E15, E16, E23, E25 and many other evidence files.
>
> Each time an evidence file uses it to choose between options is one round of leakage. There is currently no mechanism counting those rounds.

III.5.4. More strata means more false alarms

III.2.2 recommended splitting the golden set into strata, one Wilson each. Correct — but it has a consequence.

Each 95% interval carries a 5% chance of missing. More strata → the probability that **at least one** misses rises quickly:

```
strata K   P(at least 1 false alarm)
       1              5.0%
       3             14.3%
       5             22.6%
      10             40.1%
      20             64.2%
      50             92.3%
```

20 strata → a 64% chance that at least one "looks bad" purely by luck. You will go fix something that is not broken.

The remedy is Bonferroni (divide the error rate by K):

```
K= 1: conf=0.9500  Wilson(17/20)=[0.6396, 0.9476]  w=0.3081
K= 5: conf=0.9900  Wilson(17/20)=[0.5644, 0.9612]  w=0.3968
K=10: conf=0.9950  Wilson(17/20)=[0.5370, 0.9651]  w=0.4281
K=20: conf=0.9975  Wilson(17/20)=[0.5121, 0.9684]  w=0.4563
```

The price: wider intervals. But without it the report fills with false alarms.

An internal tension, and how to resolve it. III.2.2 says "*stratify so the aggregate does not hide breakage*". III.5.4 says "*stratifying raises false alarms*". Both are correct.

Resolution: stratify by **business-meaningful data types**, not by slicing finely to hunt; and apply Bonferroni when K ≥ 5.

III.5.5. The garden of forking paths

The most insidious variant of III.5.2. You are not "selecting" anything — you are merely **reporting what is interesting**.

Run probes across several dimensions (by stratum, by model, by prompt, by file type…), then write up whichever dimension produced something worth saying:

```
dimensions tried   average inflation
               1        +0.0006
               2        +0.0362
               4        +0.0642
               8        +0.0854
              16        +0.1028
```

(`seed=5`, n=30 per dimension, true ability 0.85)

Try 8 dimensions and report the best-looking one → inflation of 0.085, **while feeling entirely honest**. You hid nothing; you simply narrated the standout.

Rule: record **every** dimension tried, including the uninteresting ones.

E23 did this correctly. They recorded the Gemini failure (revoked API key) rather than silently dropping it, and stated explicitly "*therefore there is no 'small sample on a second engine' as originally planned — scope was cut to exactly 1 engine*".

That is precisely how you defeat the forking-paths trap: report what did not work too.

III.5.6. Golden set is not production

The final trap, and the one **no formula in this document can touch**.

Everything from Part II through III.5.5 measures uncertainty from **small samples** or **process**. None of it measures uncertainty from **biased sampling**.

If your golden set is 100 cleanly photographed images, Wilson yields a very tight, very pretty interval — but that interval speaks only about clean photographs. Production has blur, skew, and third-generation photocopies.

A tight interval on a biased golden set is **more dangerous** than no interval at all, because it creates the impression of careful measurement.

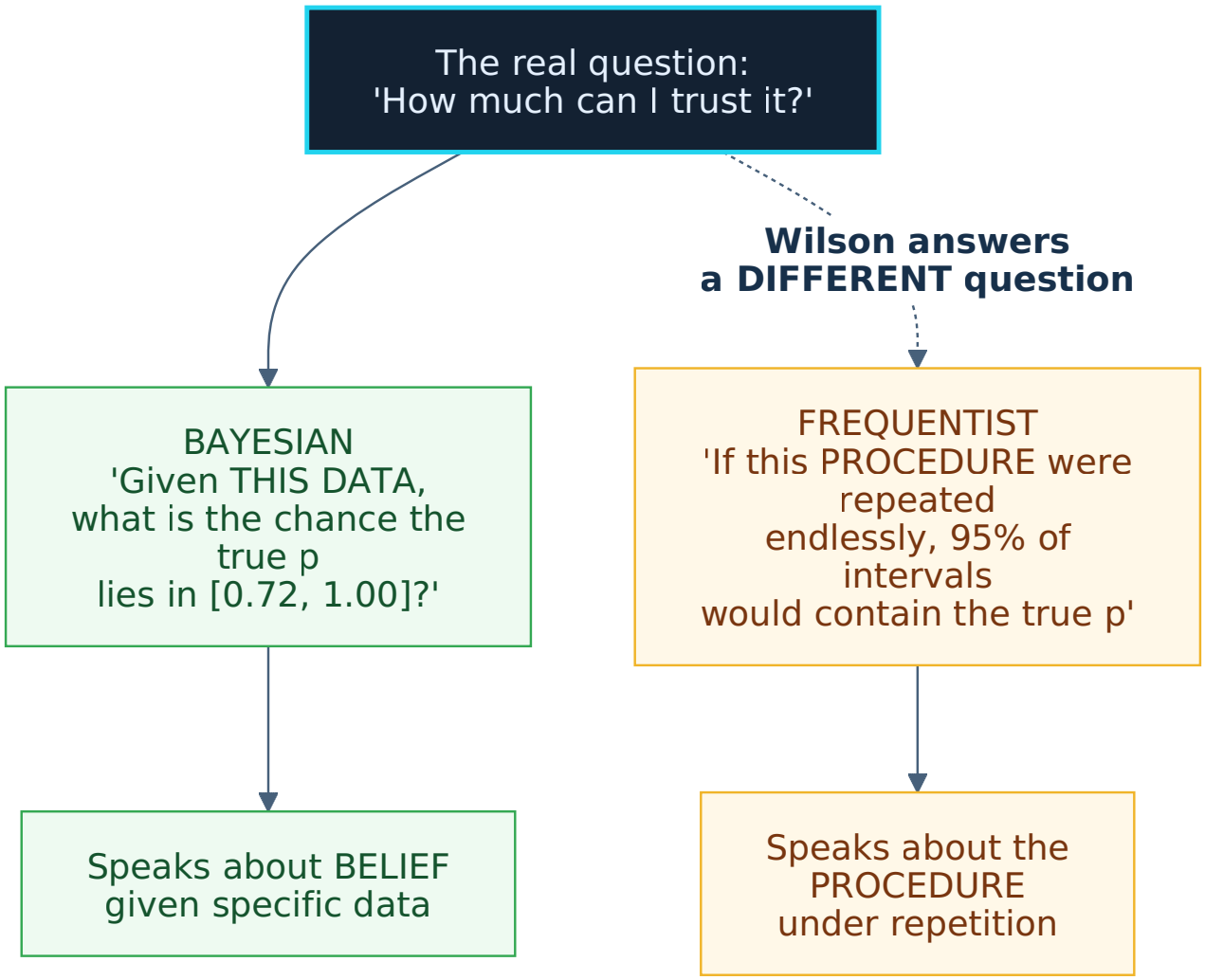
Rule: every golden set must document where its samples came from, plus one honest sentence: *how it differs from production*. If you cannot write that sentence, you do not yet know.

III.6 — The parallel Bayesian track

III.6.1. Wilson answers a DIFFERENT question than yours

The question opening this document is: "*looking at this test result, how much can I trust it?*"

Wilson **does not answer that**. It answers a different question that merely sounds similar.



Analogy: you ask a doctor "*do I have this disease?*". The doctor answers "*my test is correct 95% of the time*". A true and useful answer, but **not the question you asked**. You asked about you; the doctor answered about the test.

III.6.2. The wrong sentence everyone says

```
WRONG:   "there is a 95% probability the true p lies in [0.72, 1.00]"
CORRECT: "this procedure, if repeated, produces an interval containing the true p 95% of the
time"
```

Every section above interprets Wilson in the WRONG form ("*guarantees only ≥72%*"). For practical purposes that is acceptable — but if the harness turns this into a rule, it must know the phrasing is technically loose.

More importantly: **there is a tool that answers your actual question**.

III.6.3. The Jeffreys interval

The most common Bayesian interval for a rate is the **Jeffreys interval**, based on the **Beta** distribution:

```
posterior = Beta(k + 0.5, n − k + 0.5)

lower = the 2.5th  percentile of that distribution
upper = the 97.5th percentile of that distribution
```

The `0.5` is the **Jeffreys prior** — a neutral way of saying "I knew nothing beforehand".

III.6.4. The side-by-side table

```
      k/n            Wilson95            Jeffreys95   lower diff   upper diff
    3/3    [0.4385,1.0000]   [0.4644,1.0000]      +0.0259      +0.0000
    5/5    [0.5655,1.0000]   [0.6206,1.0000]      +0.0551      +0.0000
   8/10    [0.4902,0.9433]   [0.4972,0.9559]      +0.0071      +0.0126
   9/10    [0.5958,0.9821]   [0.6187,0.9890]      +0.0228      +0.0069
  10/10    [0.7225,1.0000]   [0.7828,1.0000]      +0.0603      +0.0000
   6/10    [0.3127,0.8318]   [0.3037,0.8469]      −0.0090      +0.0151
  20/20    [0.8389,1.0000]   [0.8834,1.0000]      +0.0445      +0.0000
  27/30    [0.7438,0.9654]   [0.7566,0.9710]      +0.0128      +0.0056
  30/30    [0.8865,1.0000]   [0.9203,1.0000]      +0.0338      +0.0000
  48/50    [0.8654,0.9890]   [0.8778,0.9916]      +0.0124      +0.0026
 88/100    [0.8019,0.9300]   [0.8057,0.9327]      +0.0038      +0.0027
 96/100    [0.9016,0.9843]   [0.9077,0.9864]      +0.0061      +0.0020
100/100    [0.9630,1.0000]   [0.9753,1.0000]      +0.0122      +0.0000
   0/30    [0.0000,0.1135]   [0.0000,0.0798]      +0.0000      −0.0337
   1/20    [0.0089,0.2361]   [0.0054,0.2108]      −0.0034      −0.0253
 70/100    [0.6042,0.7811]   [0.6055,0.7832]      +0.0013      +0.0021
```

Three observations:

(a) At large n they nearly coincide. `70/100` differs by 0.0013 — negligible.

(b) At small-n all-pass, the gap is clear. `10/10` differs by **0.0603**, `20/20` by **0.0445**. Jeffreys is **more generous** at the lower bound. This is exactly where probe-first lives.

(c) At zero-failure, Jeffreys is TIGHTER. `0/30` → Wilson says the error rate could reach 0.1135, Jeffreys says 0.0798.

III.6.5. Sample savings

Minimum all-pass samples for the lower bound to reach T:

```
T=0.80:  Wilson= 16   Jeffreys= 11    (saves   5 samples)
T=0.90:  Wilson= 35   Jeffreys= 24    (saves  11 samples)
T=0.95:  Wilson= 73   Jeffreys= 49    (saves  24 samples)
T=0.99:  Wilson=381   Jeffreys=250    (saves 131 samples)
```

At T=0.99, Jeffreys saves **131 samples** — with an expensive probe like E14 (23.8 s/page) that is a real cost: 131 × 23.8 s ≈ **52 minutes**.

But read alongside III.1.6: Jeffreys carries **no** coverage guarantee like Clopper-Pearson. It is "cheaper" because it answers a different question.

III.6.6. The ship question: posterior probability

This is where the Bayesian track shows its clearest value.

The real decision question is not "*what is the interval?*" but: **What is the probability this pipeline achieves ≥ 90%?**

Bayesian answers directly: `P(p ≥ T | data)`.

```
      k/n   Wilson lower bound   P(p≥0.90)   P(p≥0.95)   P(p≥0.99)
   10/10             0.7225         0.8584      0.6949      0.3502
   20/20             0.8389         0.9612      0.8505      0.4765
   30/30             0.8865         0.9884      0.9219      0.5645
   50/50             0.9287         0.9989      0.9768      0.6851
  100/100            0.9630         1.0000      0.9987      0.8443
   96/100            0.9016         0.9859      0.6558      0.0081
   92/100            0.8500         0.7393      0.0910      0.0000
   46/50             0.8116         0.6624      0.1611      0.0005
  185/200            0.8800         0.8838      0.0593      0.0000
    9/10             0.5958         0.4385      0.1986      0.0217
```

This table reads far more naturally than intervals. `30/30` → "*98.84% chance of achieving ≥90%*" is more direct than "*the lower bound is 0.8865*".

And the two approaches disagree on decisions. At threshold 0.90:

```
30/30:  Wilson lower bound = 0.8865 < 0.90  →  FAIL
        P(p ≥ 0.90) = 0.9884                →  PASS
```

Thirty out of thirty passing, and Wilson still blocks. Bayesian clears it at 98.84% confidence.

"Wilson lower bound ≥ 0.90" is STRICTER than "P(p≥0.90) ≥ 0.90". If the harness gates on Wilson, it will block a lot of genuinely good work.

III.6.7. A decision threshold is not an interval

More broadly than III.6.6: both approaches ignore **the cost of the two error types**.

```
shipping something bad   → cost C1
blocking something good  → cost C2
```

If C1 = 10 × C2, the sensible threshold differs from the C1 = C2 case. No interval formula has a place for that information.

Bayesian at least **has a framework** for incorporating cost. Frequentist does not.

This document does not pursue that direction — it requires (C1, C2) estimates that almost no project can produce. But it is worth knowing that "what should the threshold be" **has no purely statistical answer**.

III.6.8. Priors — reusing evidence you already have

This is what Bayesian can do and frequentist **cannot**: incorporate prior knowledge.

A new probe: 8/10 pass. The conclusion depends on what you knew **beforehand**:

prior	Beta(a,b)	95% interval	width
nothing (Jeffreys prior)	Beta(8.5, 2.5)	[0.4972, 0.9559]	0.4587
previously saw 20/25	Beta(28.0, 7.0)	[0.6547, 0.9130]	0.2583
previously saw 90/100	Beta(98.0, 12.0)	[0.8266, 0.9418]	0.1152
previously saw 450/500	Beta(458.0, 52.0)	[0.8704, 0.9228]	0.0524

Same 8/10 probe, an interval **8.8× narrower** when you hold prior data.

Why this matters for the OCR repo. `address_text_scan_dongpv` holds **E11 through E45** — dozens of evidence files run against the same 100 pages, the same 6 case files.

When running a new probe you are **not** in a state of knowing nothing. Wilson forces you to pretend you are, discarding everything accumulated.

This maps directly onto how probe-first operates: **evidence accumulating over time**.

III.6.9. A prior is a double-edged blade

A wrong prior makes you confidently wrong, and it takes **a very long time to correct**.

Simulation: the truth is p = 0.60. The prior claims "450/500 = 0.90" (badly wrong). A new probe of n samples:

```
n=   10: posterior=[0.8660, 0.9193]  does NOT contain 0.60
n=   30: posterior=[0.8544, 0.9090]  does NOT contain 0.60
n=  100: posterior=[0.8204, 0.8774]  does NOT contain 0.60
n=  300: posterior=[0.7585, 0.8151]  does NOT contain 0.60
n= 1000: posterior=[0.6766, 0.7229]  does NOT contain 0.60
n= 3000: posterior=[0.6269, 0.6587]  does NOT contain 0.60
```

Even 3000 samples have not pulled it back to the truth.

By comparison: Wilson (no prior) at n=100 gives `[0.5020, 0.6906]` — which **does** contain 0.60. Without a prior you converge more slowly but you are never led astray.

III.6.10. Weighted priors — the safe way to use them

The remedy: **down-weight** old evidence, because conditions have changed.

New probe 8/10, old evidence 90/100, discounted by factor w:

```
w=1.0  (old evidence ~90/100 = 100 samples):  [0.8226, 0.9390]  width=0.1163
w=0.5  (old evidence ~45/50  =  50 samples):  [0.7846, 0.9463]  width=0.1617
w=0.2  (old evidence ~18/20  =  20 samples):  [0.7135, 0.9533]  width=0.2398
w=0.1  (old evidence ~9/10   =  10 samples):  [0.6514, 0.9559]  width=0.3044
w=0.0  (no prior evidence used):              [0.4972, 0.9559]  width=0.4587
```

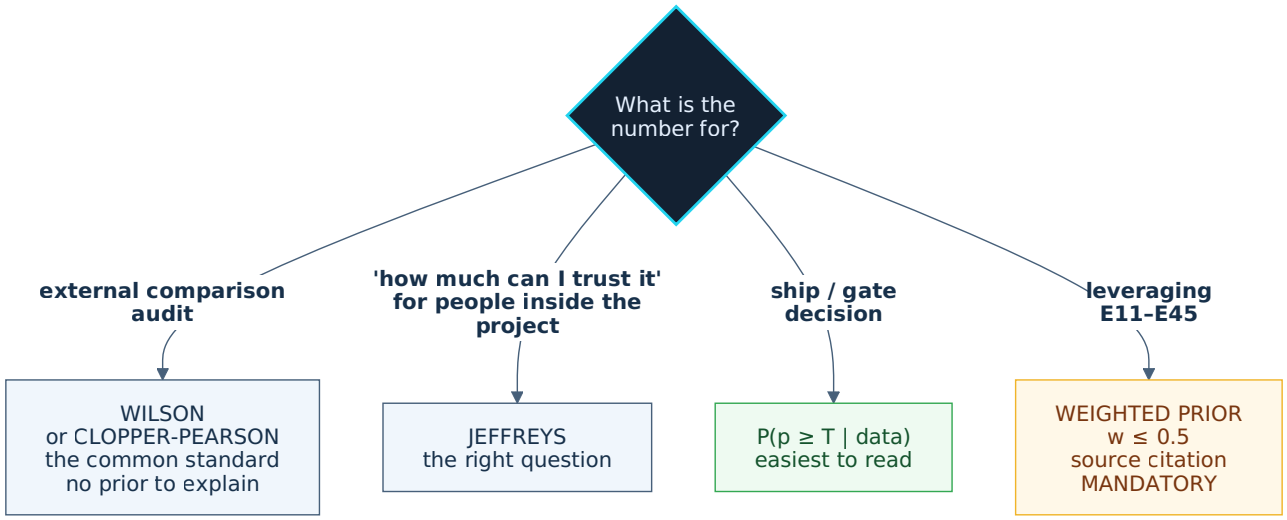
`w` has an intuitive reading: "*how many samples the old evidence is worth under current conditions*".

Old evidence versus current conditions	w
Identical (same model, same data type, same week)	0.5
Close (one small factor changed)	0.2
Clearly different (model or data type changed)	0.1
Entirely different, or unsure	0.0 — no prior

Never use w = 1.0. Conditions are never identical, and III.6.9 shows the cost of being wrong is too high.

(These four levels are a judgement call, not derived from data — see IV.10 Q9.)

III.6.11. Use both — the rule



The two tracks answer two different questions, so report **both**.

> **A mandatory constraint when using priors:** you must record a **link to the source evidence** and the **weight w with its justification**. A prior without provenance is an invented prior, and III.6.9 shows an invented prior outweighs 3000 samples of real data.

PART IV

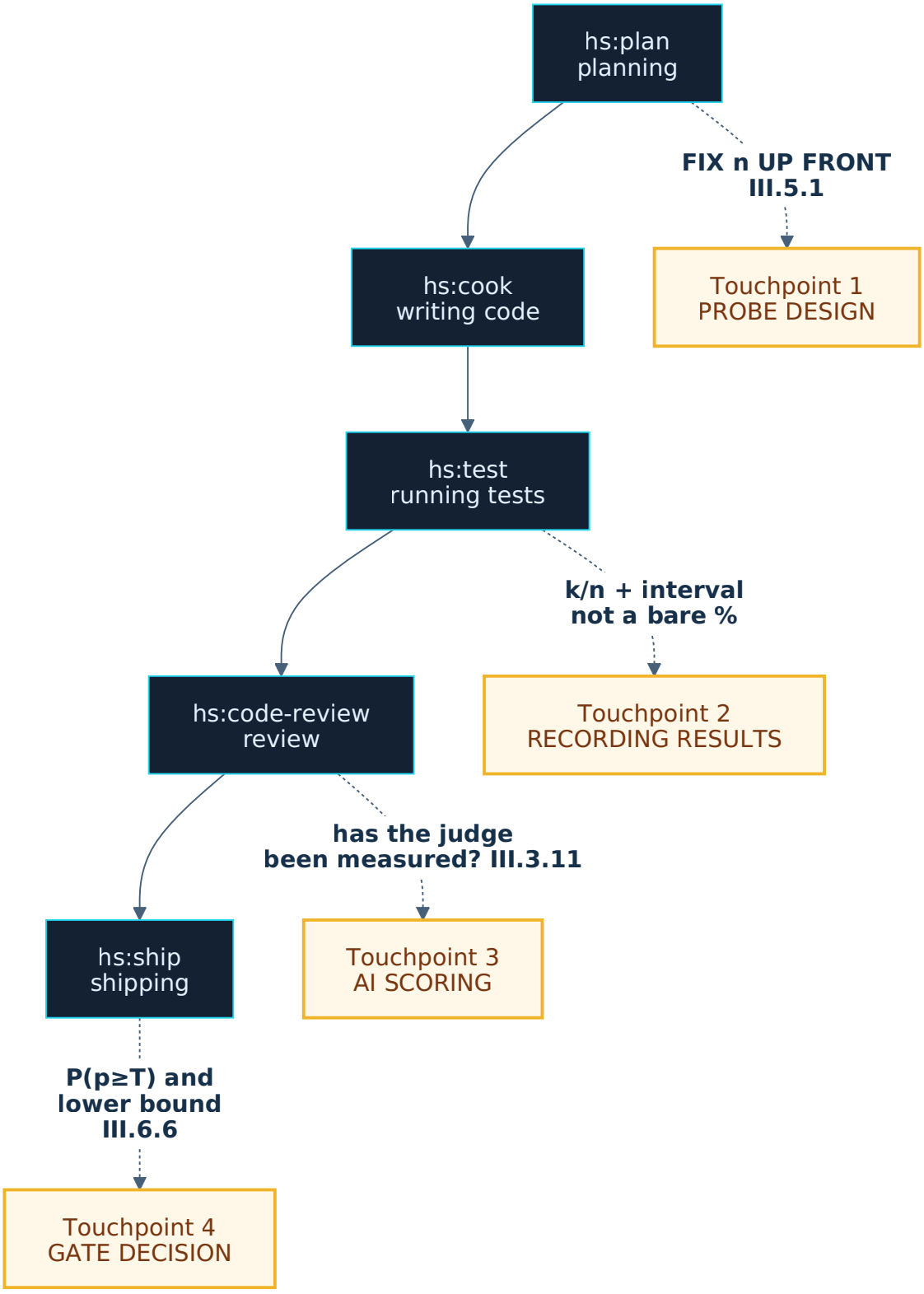
PRACTICE

Back to reality. The verification phase in an ordinary working day — especially when most of the work is done by AI and scored by AI.

IV.1 — What the verification phase looks like

The three parts above supply the tools. This part says **when to reach for which**, inside the real working rhythm.

A working day with the harness typically runs `plan → cook → test → review → ship`. Confidence intervals touch four points in that chain.



Four touchpoints, four sections.

IV.2 — Touchpoint 1: probe design (inside `hs:plan`)

Done **before running** the first command. The cheapest step, and it blocks the most expensive trap.

IV.2.1. Three questions to answer first

Question	Why	See
What is n? Fixed up front, not adjusted mid-run.	Blocks optional stopping — 10× the false-claim rate	III.5.1
How many independent sources? Not how many samples.	The hard ceiling `n_eff ≤ K/ICC`	III.2.7
What is threshold T? Then check the table whether n suffices.	30 samples can never demonstrate ≥90%	II.6

IV.2.2. The sample-size table — pin it to the wall

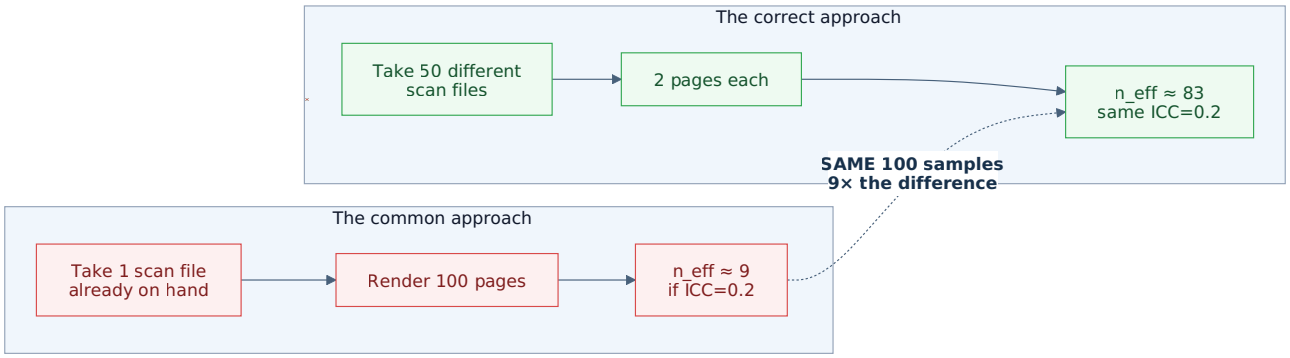
All-pass (`k = n`), Wilson 95%, the maximum claim permitted:

```
  1/1     →  ≥ 20.7%     (nearly meaningless)
  3/3     →  ≥ 43.9%
  5/5     →  ≥ 56.6%
 10/10    →  ≥ 72.2%
 20/20    →  ≥ 83.9%
 30/30    →  ≥ 88.6%
 50/50    →  ≥ 92.9%
 73/73    →  ≥ 95.0%
100/100   →  ≥ 96.3%
200/200   →  ≥ 98.1%
381/381   →  ≥ 99.0%
500/500   →  ≥ 99.2%
```

With **failures**, to keep the lower bound ≥ 0.90:

```
n =  30  →  unreachable, even at 30/30
n =  50  →  at most  0 failures
n = 100  →  at most  4 failures
n = 200  →  at most 11 failures
n = 500  →  at most 36 failures
```

IV.2.3. Sample design: sources matter more than volume



And if the real dataset contains no negative cases (as in E14: no rotated pages), you must **deliberately construct them** — E14 hand-rotated 20 pages on a balanced schedule, `seed=42`. See III.3.8.

IV.2.4. Recording it in the plan

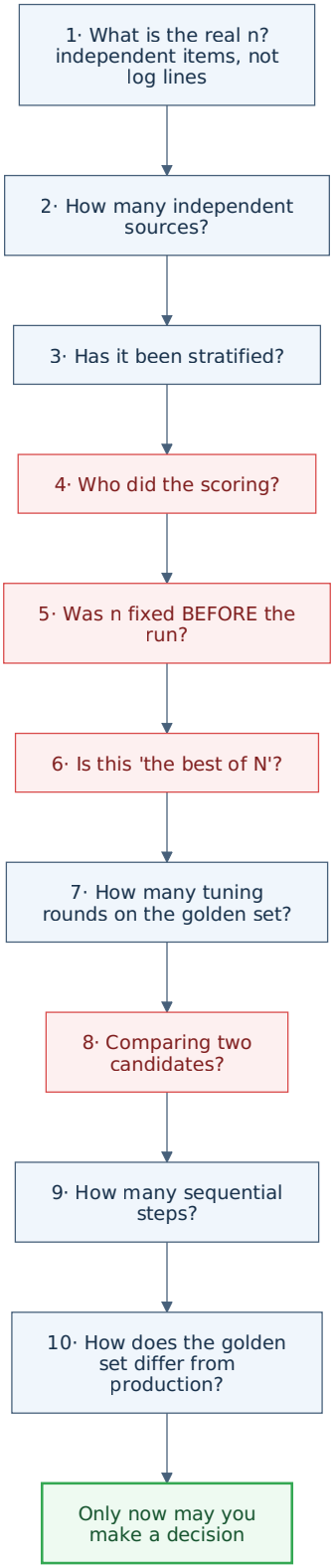
Three lines, added to the plan's verification phase:

```
probe-design:
  n: 100                    # fixed up front, not adjusted
  sources: 25               # independent source count (K)
  threshold: 0.90           # committed threshold
  route: cluster-floor      # K=25 ≥ 20 → actually uses icc
  peek: none                # no mid-run looks
```

IV.3 — Touchpoint 2: recording results (inside `hs:test`)

IV.3.1. Ten questions before trusting a number

Ask them in order. Fail any one and the number is not ready to drive a decision.



1. What is the real `n`? Count **independent items**, not log lines, not runs.

The trap: 50 items re-run 10 times gives 500 log lines, but `n = 50`.

```
n = 50   (46/50)     Wilson95 = [0.8116, 0.9685]   width = 0.1568
n = 500  (460/500)   Wilson95 = [0.8929, 0.9407]   width = 0.0478   ← WRONG
```

The 10 repetitions are still useful, but they measure something **else**: run-to-run stability. Report that separately; do not fold it into n.

2. How many independent sources? → III.2 K ≥ 20 → estimate ICC. K < 10 → cluster floor. If uncorrected, state "*cluster-uncorrected*".

3. Has it been stratified? → III.2.2 Wilson per data type. The decision number is the **worst stratum**. K ≥ 5 strata → Bonferroni.

4. Who did the scoring? → III.3 Human → go to question 5. AI → has the judge's confusion matrix been measured on ≥50 balanced human-scored samples? If not, label `[ASSUMED: judge uncalibrated]` and stop.

5. Was `n` fixed BEFORE the run? → III.5.1

6. Is this number "the best of several"? → III.5.2, III.5.5 If so: select on set A, report on set B. Record **every** candidate tried.

7. How many tuning rounds has the golden set seen? → III.5.3

8. Comparing two candidates? → III.4 Same test set → McNemar. Different sets → difference interval. **Never** eyeball overlapping intervals.

9. How many sequential steps? → III.3.13 Multiply the probabilities. Two steps on the same model → **not** independent verification.

10. How does the golden set differ from production? → III.5.6 Write one sentence. If you cannot, you do not yet know.

IV.3.2. How to write it in the report

Instead of:

```
[OBSERVED] pipeline achieves 92% accuracy
```

Write:

```
[OBSERVED] golden-100, human-scored, n fixed before the run
  design:         5 strata × 20, 47 independent sources (K=47 → route icc)
  total:          92/100   Wilson95 = [0.8500, 0.9589]
  worst stratum:  17/20    Wilson95 = [0.6396, 0.9476]  (handwriting)
                           Bonferroni K=5 → [0.5644, 0.9612]
  → guarantees only ≥ 56% on handwriting (after multi-stratum correction)
  → P(p ≥ 0.90 | data) = 0.7393   [Jeffreys prior, no prior evidence used]
  → golden set lacks: phone photographs, aged yellowed documents
  → golden set has seen 3 tuning rounds
```

Considerably longer. But the first line is marketing; the rest is engineering.

IV.3.3. Three levels of application

Not every probe needs all ten questions.

Level	When	Questions required
1 · Exploratory (light)	Discovery probe, nothing built on it yet	Questions 1 and 10. Record `k/n` + Wilson.
2 · Report-grade (standard)	Someone will read and believe it	Questions 1–4, 8, 10. Add stratification and cluster correction.
3 · Gate-grade (full)	An irreversible decision	All ten. Add tier-2 judge calibration, `P(p ≥ T)`, and prior provenance if used.

The purpose of three levels is so level 3 does not discourage level 1 — exploratory probes must stay cheap.

IV.4 — Touchpoint 3: when AI does the scoring

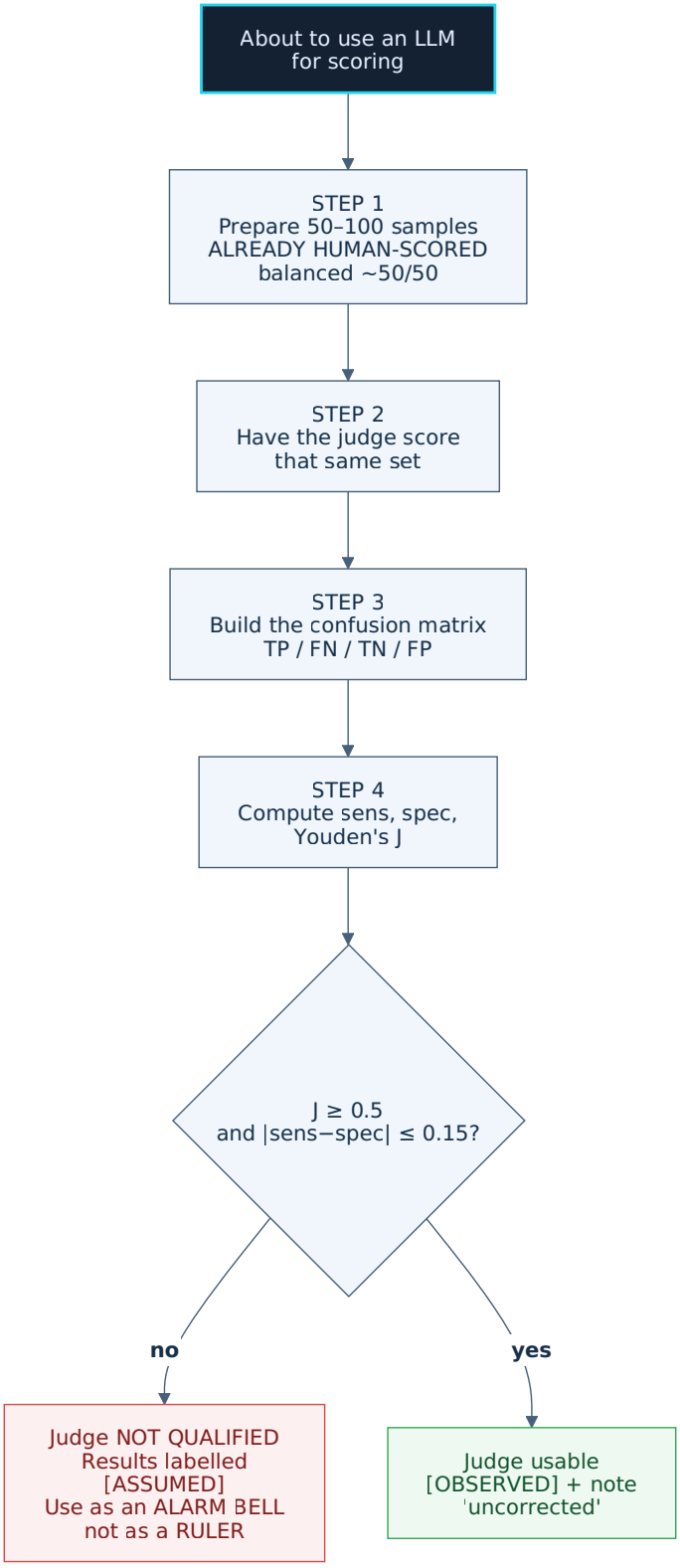
This is the touchpoint most specific to working with AI, and the one most often skipped.

IV.4.1. Recognising you are in this situation

Four signs, any one suffices:

- Pass/fail results are produced by an LLM (`hs:code-review`, `hs:critique`, the `hs:red-teamer` agent…)
- One LLM checks another LLM's output
- You read an agent's conclusion and write it straight into a report
- The judge and the pipeline use **the same model**

IV.4.2. The judge calibration procedure



IV.4.3. A full worked example: recalibrating E23's judge

This is an applied exercise on real data.

Current state (III.3.6): E23's judge has `J = 0.4650`, `n_calib = 35` → fails both criteria.

What it would take to pass:

Task	Detail	Estimated cost
Raise `n_calib` to ≥ 50	Add 15 more human-scored pages	~15 pages of viewing time
Balance the calibration set	Currently 13 wrong / 22 right ≈ 37/63. Needs ~50/50 → add software-wrong cases	Must **actively hunt** wrong cases
Raise J to ≥ 0.5	Needs higher sens or spec → revise the judge prompt	A few prompt iterations
Diversify sources	6 case files is far too few (III.2.7) — ceiling `n_eff ≤ 20` at ICC=0.3	Requires new case files

The highest-value task first: raise J. Because at `1/J = 2.15`, all correction is meaningless.

Look at the 5 "LLM false rejection" cases E23 already classified:

E23's classification	Count	Fixable by prompt?
Self-contradiction (reasoning says RIGHT, label says WRONG)	1	**Yes** — require the label to match the reasoning
Semantic dispute (the LLM has a point)	1	No — requires fixing the **ground truth**
Scored a field outside the rubric	1	**Yes** — state explicitly that only 3 fields are scored
Genuine LLM error (confused "ledger number" with serial)	2	**Yes** — give discriminating examples in the prompt

Four of the five are fixable by prompt. If all four are fixed, `FP` drops from 5 to 1:

```
new spec = 21/22 = 0.9545
new J    = 0.6923 + 0.9545 − 1 = 0.6468   → PASSES (> 0.5)
new 1/J  = 1.55  (instead of 2.15)
```

This is immediately actionable and derives entirely from data E23 already holds — no new runs required.

IV.4.4. When the judge fails: still usable, used differently

A judge failing the gate does NOT mean discarding it. It means changing how you use it:

Judge passes the gate	Judge fails the gate
Use as a **ruler** — its number goes in the report	Use as an **alarm** — summon a human
Results `[OBSERVED]`	Results `[ASSUMED]` need human confirmation
May be back-corrected (tier 2)	No correction — `1/J` too large

E23 used it exactly the second way: *"9 pages triggered the bell, catching 9 of 13 real errors"*.

IV.5 — Touchpoint 4: the gate decision (inside `hs:ship`)

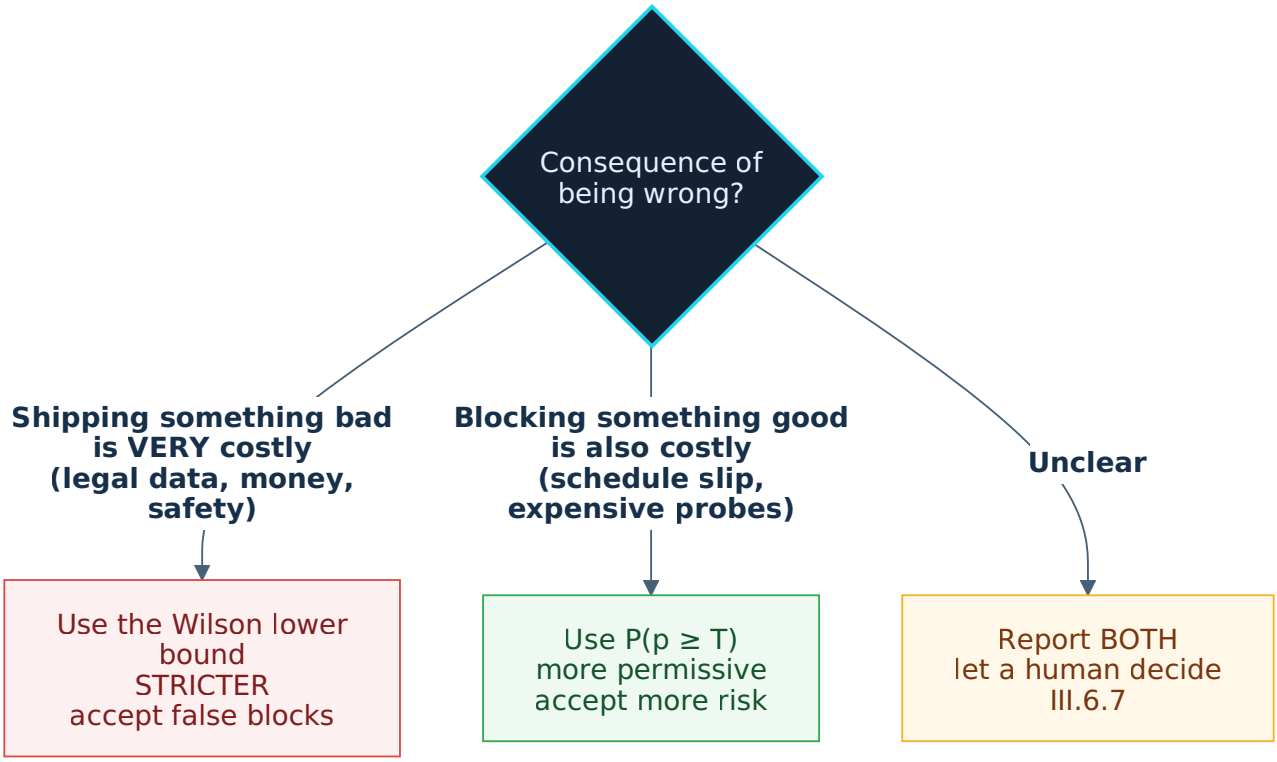
IV.5.1. Two numbers, not one

At the decision point, report **both**:

```
Wilson lower bound = 0.8865      ← compare to threshold, the common standard
P(p ≥ 0.90)        = 0.9884      ← readable, answers the real question
```

And know they may **disagree** (III.6.6): `30/30` gives Wilson = FAIL but `P(p≥0.90) = 0.9884` = PASS.

IV.5.2. Choosing the criterion



For the OCR repo — the data is legal documentation, and misreading a serial number is a serious error — the left branch is correct.

IV.5.3. Flags to check before a PASS

Four flags; any one present means the gate must not pass silently:

Flag	Meaning	See
`saturated`	The interval touches 0 or 1 — overflowed, not narrow	III.3.10
`route = cluster-floor`	Too few independent sources, using the most conservative path	III.2.10
`judge = rejected` or `biased`	The ruler is not qualified	III.3.11
`prior_used`	Old evidence was used — must cite source and w	III.6.10

IV.6 — Applied backwards: what to fix in the two repos

Consolidating everything into concrete tasks.

IV.6.1. OCR repo — `address_text_scan_dongpv`

#	Task	Why	Cost
1	**Add case files, not pages**	Hard ceiling `n_eff ≤ K/ICC`; 6 files caps you around 20 effective samples	High — needs new data
2	**Fix the E23 judge prompt per IV.4.3**	4 of 5 false rejections are prompt-fixable → J from 0.4650 to ~0.6468	Low — a few iterations
3	**Record the case-file count in every evidence file**	So ICC is computable; currently it must be inferred	Very low
4	**Count the golden set's tuning rounds**	E11 has served as the ruler for E14, E15, E16, E23, E25… — leakage accumulates	Low
5	**Document how E11 differs from production**	100 pages from 6 case files is a very narrow sample	Low

Task 1 is the most important and the most expensive. Stated plainly: every other improvement only makes the number more honest, not more **certain**. Only new case files achieve that.

IV.6.2. Harness repo — `sdlc-harness`

#	Task	Why	Cost
1	**Write `harness/scripts/wilson.py`**	The foundation for everything else	Medium — spec in IV.7
2	**Replace `rel_band` in `hs:bakeoff`**	The 0.05 constant → McNemar / difference interval	Low once (1) exists
3	**Add a held-out requirement for bake-offs**	Winner's curse inflates by +0.10	Low
4	**Require `[OBSERVED]` rates to carry k/n + interval**	Distinguishes 3/3 from 300/300	Low — a rule plus a nudge
5	**A mandatory judge-calibration procedure**	Gates currently measure through an uncalibrated scale	**High** — needs 50–100 human-scored samples per judge
6	**Probe metadata: n fixed up front, candidate count, tuning rounds**	Blocks the three most dangerous process traps	Low — three lines

Task 6 is the cheapest and blocks the most. Three metadata lines block optional stopping, the winner's curse, and golden-set leakage — the three traps III.5 shows to be more dangerous than the entire continuity-correction debate.

Task 5 may die on cost. Stated up front to avoid surprise: if you are not prepared to fund 50–100 human-scored samples per judge, apply it **only to hard gates** and exempt advisory judges.

IV.7 — `wilson.py` specification

This is a specification, not code. Input for `hs:plan` / `hs:cook`.

IV.7.1. Location and constraints

- Path: `harness/scripts/wilson.py`
- **Pure Python standard library.** Only `math`, `statistics.NormalDist`, `argparse`, `json`. **No** dependency on `scipy` / `numpy` — the harness must run in repos lacking them.
- No state, no network. Pure functions plus a CLI.
- Clopper-Pearson and Beta quantiles require bisection over the binomial: use `math.lgamma` to avoid overflow, and **subtract `logB` inside the log** rather than dividing afterwards.

> **This bug was hit for real while writing this document.** Dividing afterwards (`integral / exp(logB)`) causes `exp(logB)` to underflow to 0 for large parameters (a+b ≥ 500) → `ZeroDivisionError`. Stay in log space: `exp(log_integrand − logB)`.
>
> This is a mandatory test case, not an optional detail.

IV.7.2. API — basic intervals

```
def interval(k, n, conf=0.95, method="wilson") -> tuple[float, float]
    # method: "wilson" | "wilson-cc" | "clopper-pearson" | "jeffreys"

def interval_full(k, n, conf=0.95, method="wilson") -> dict
    # -> {p_hat, z, center, half_width, lower, upper, width, conf, n, k, method}

def coverage(n, p, conf=0.95, method="wilson") -> float
    # sums EXACTLY over k = 0..n. No simulation. Used in tests.

def min_n_all_pass(target, conf=0.95, method="wilson") -> int
def max_fails(n, target, conf=0.95, method="wilson") -> int | None
```

IV.7.3. API — clusters

```
def icc_anova(clusters: list[tuple[int, int]]) -> dict | None
    # -> {icc, n0, msb, msw, K, N}  ; None when K < 2

def cluster_adjusted(clusters, conf=0.95) -> dict
    # picks the route by K:
    #   K >= 20  -> "icc"            (point estimate)
    #   10<=K<20 -> "icc-upper"      (upper bound, conservative)
    #   K < 10   -> "cluster-floor"  (one sample per cluster)
    # -> {route, icc, deff, n_eff, lower, upper, K, N}
```

No user-declared `--icc` parameter — III.2.8 shows no default constant is safe. The output **must print which route was chosen**.

IV.7.4. API — comparison

```
def mcnemar_wilson(b, c, conf=0.95) -> dict
    # same test set. -> {n_disc, lower, upper, conclusive}
    # conclusive = the interval does NOT contain 0.5

def diff_newcombe(k1, n1, k2, n2, conf=0.95) -> dict
    # different test sets. -> {diff, lower, upper, conclusive}
    # conclusive = the interval does NOT contain 0
```

These two functions defend against the III.4.5 error at the tooling layer: **there is no function named anything like `intervals_overlap()`**.

IV.7.5. API — judge

```
def judge_screen(TP, FN, TN, FP, conf=0.95) -> dict
    # Tier 1 (III.3.11)
    # -> {sens, spec, sens_ci, spec_ci, youden_j, verdict, bias_direction, n_calib}
    # verdict: "ok" | "biased" | "rejected"
    #   rejected when J < 0.5, or TP+FN == 0, or TN+FP == 0
    #   biased   when |sens - spec| > 0.15

def rogan_gladen(p_obs, sens, spec) -> float | None
    # None when sens + spec <= 1

def judge_adjust(k, n, TP, FN, TN, FP, conf=0.95, split=3) -> dict
    # Tier 2 (III.3.7)
    # -> {lower, upper, raw_lower, raw_upper, saturated_low, saturated_high, verdict}
    # MUST return the raw unclamped values too — III.3.10
```

IV.7.6. API — Bayesian

```
def jeffreys(k, n, conf=0.95) -> tuple[float, float]

def posterior_prob_ge(k, n, T, prior_a=0.5, prior_b=0.5) -> float
    # P(p >= T | data)  — III.6.6

def prior_from_evidence(k_old, n_old, weight) -> tuple[float, float]
    # -> (a, b) for the Beta prior, already down-weighted
    # a = 0.5 + k_old*weight ;  b = 0.5 + (n_old-k_old)*weight
    # RAISES if weight > 0.5 — III.6.10 forbids w = 1.0
```

`prior_from_evidence` raising above `weight > 0.5` is **deliberate**: it turns the III.6.10 rule into a constraint the tool will not let you violate.

IV.7.7. CLI

```
# basic interval
wilson.py --k 8 --n 10
wilson.py --k 8 --n 10 --conf 0.90 --method clopper-pearson --json

# gate: exit 0 if lower >= threshold, exit 1 otherwise
wilson.py --k 96 --n 100 --threshold 0.90

# sample size
wilson.py --min-n 0.90
wilson.py --max-fails --n 200 --threshold 0.90

# clusters
wilson.py --clusters clusters.json

# strata (auto-Bonferroni when K >= 5)
wilson.py --strata strata.json

# comparison
wilson.py --mcnemar --b 12 --c 8
wilson.py --diff --k1 90 --n1 100 --k2 80 --n2 100

# judge
wilson.py --judge-screen --tp 34 --fn 2 --tn 14 --fp 0
wilson.py --judge-adjust --k 88 --n 100 --tp 34 --fn 2 --tn 14 --fp 0

# Bayesian
wilson.py --k 30 --n 30 --bayes
wilson.py --k 30 --n 30 --prob-ge 0.90
wilson.py --k 8 --n 10 --prior-k 90 --prior-n 100 --prior-weight 0.2
```

JSON file formats:

```
// clusters.json
{"clusters": [{"name": "CV234036", "k": 18, "n": 20}, ...]}

// strata.json
{"strata": [{"name": "handwriting", "k": 17, "n": 20}, ...]}
```

IV.7.8. Exit codes

Code	Meaning
0	computed successfully; with `--threshold`, `lower >= threshold`
1	`--threshold` given and `lower < threshold` — **not met**
2	invalid input (validation error)
3	judge **rejected** (III.3.11) — the result does not qualify as `[OBSERVED]`

IV.7.9. Output format

Default — human-readable:

```
k=8  n=10  conf=95%  method=wilson
p_hat  = 0.8000
center = 0.7167   (pulled toward 0.5 by small n)
CI     = [0.4902, 0.9433]   width = 0.4532
```

`--json` — machine-readable, **no rounding**:


```
{"k": 8, "n": 10, "conf": 0.95, "method": "wilson", "p_hat": 0.8,
 "z": 1.959963984540054, "center": 0.7167397386975674,
 "half_width": 0.22657830205783206,
 "lower": 0.49016143663973537, "upper": 0.9433180407552995,
 "width": 0.45315660411566413}
```

IV.7.10. Mandatory edge-case tests

Case	Expected behaviour
`n = 0`	return `(0.0, 1.0)`. No division by zero, no raise.
`k = 0`	lower = exactly 0.0, upper > 0
`k = n`	lower < 1.0, upper = exactly 1.0
`k > n`, `k < 0`, `n < 0`	raise `ValueError`
`conf` outside `(0, 1)`	raise `ValueError`
`k`, `n` not integers	raise `TypeError`
boundary clamping	lower never < 0, upper never > 1
Clopper-Pearson at n ≥ 500	no overflow thanks to `lgamma`; < 100 ms via caching
Beta quantile with a+b ≥ 500	no `ZeroDivisionError` — subtract `logB` inside the log
`icc_anova` with K < 2	return `None`, do not raise
`icc_anova` when `p̄ = 0` or `1`	ICC = 0.0
`icc_anova` when MSB < MSW	ICC clamps to 0.0, never negative
`cluster_adjusted` with K = 5	route = `"cluster-floor"`, NOT ICC
`mcnemar_wilson(0, 0)`	`n_disc = 0` → `conclusive = False`, interval `(0.0, 1.0)`
`judge_screen` with `TP+FN = 0` or `TN+FP = 0`	verdict `"rejected"`
`judge_adjust` when `J <= 0`	return `[0, 1]` + verdict `"rejected"`, no division by zero
`judge_adjust` when raw exceeds 1.0	`saturated_high = True`, `raw_upper` keeps the raw value
`prior_from_evidence(90, 100, 1.0)`	raise `ValueError`
Rogan-Gladen round trip	`rg(p·s + (1−p)(1−sp), s, sp) == p` to 1e-9

IV.7.11. Regression anchors

From the 2026-08-04 probe:

```
interval(8, 10, 0.95, "wilson")            == (0.490162, 0.943318)
interval(8, 10, 0.95, "wilson-cc")         == (0.442200, 0.964600)
interval(100,100,0.95,"clopper-pearson")[0] == 0.963800   # narrower than wilson 0.963000
interval(10, 10, 0.95, "jeffreys")[0]      == 0.782804
interval(30, 30, 0.95, "jeffreys")[0]      == 0.920322
interval(0,  30, 0.95, "jeffreys")[1]      == 0.079681

coverage(10, 0.99, 0.95, "wilson")         == 0.9044  (±1e-4)
coverage(10, 0.99, 0.95, "wilson-cc")      == 0.9957  (±1e-4)
coverage(10, 0.30, 0.95, "wilson")         == 0.924403 (±1e-5)

icc_anova([(18,20),(18,20),(19,20),(18,20),(17,20)])["icc"] == 0.0
icc_anova([(20,20),(20,20),(19,20),(8,20),(5,20)])["icc"]   ≈ 0.5619

min_n_all_pass(0.95, method="wilson")          == 73
min_n_all_pass(0.95, method="clopper-pearson") == 72
min_n_all_pass(0.95, method="jeffreys")        == 49

diff_newcombe(90, 100, 80, 100)  → lower ≈ +0.0001, conclusive = True
posterior_prob_ge(30, 30, 0.90)  ≈ 0.9884

# Anchored on REAL data (E23) — use as an integration test:
judge_screen(TP=9, FN=4, TN=17, FP=5) → youden_j ≈ 0.4650, verdict = "rejected"
```

Self-confirmation warning. The values above were produced by this very probe, so they catch **regressions** only — they do not prove the formulas correct. Additionally required:

1. **Cross-checks against** `scipy` (skipped when scipy is absent): `scipy.stats.binomtest(...).proportion_ci(method='wilson')` and `method='exact'`.
2. **Round-trip tests** for Rogan-Gladen — no external library needed.
3. **Property tests**: monotonic in conf; symmetry of `wilson(k,n)` ↔ `wilson(n−k,n)` about 0.5; Clopper-Pearson coverage always ≥ conf.

IV.7.12. Out of scope

- Do not auto-parse test result files — this is a calculator, not a parser.
- Do not decide ICC or prior weights. The user declares the source; the script computes and warns.
- Do not wire up hooks in the same phase — make the script correct first, gate later.
- Do not implement bootstrap / profile likelihood for the combined judge interval in v1 (IV.10 Q5).

IV.8 — Five integration points in the harness

(1) Evidence labels. When a claim is a **rate**, `[OBSERVED]` is valid only with `k/n` and an interval. A bare `[OBSERVED] 92% accuracy` counts as a malformed claim.

(2) `hs:bakeoff` — a machine-checkable noise band. → III.4.6 - Same test set → Wilson on `b/n_disc` containing 0.5 = within noise. - Different sets → the difference interval containing 0 = within noise. - Forbid "overlapping intervals" as a criterion. - Additionally: bake-offs must **report on a held-out set** (III.5.2).

(3) Minimum n thresholds. The IV.2.2 table becomes a lookup rule. Replaces "run a few probes to be safe" with a number.

(4) The mandatory LLM-judge calibration. → III.3.11 Any gate or report containing LLM-scored numbers must carry a **judge calibration record**: the confusion matrix over ≥50 balanced human-scored samples. Without it, downgrade to `[ASSUMED]`.

(5) Process discipline. → III.5 Three metadata lines per probe: - was `n` fixed before the run (III.5.1) - how many candidates / dimensions were tried (III.5.2, III.5.5) - how many tuning rounds the golden set has seen (III.5.3)

The Bayesian track: how to introduce it

Decision: **run it in parallel, do not replace** (III.6.11).

- `wilson.py` supports both (IV.7.6).
- The standard report carries Wilson as the primary number, plus `P(p ≥ T)` as a secondary.
- Priors only with a **link to the source evidence** and a **weight w ≤ 0.5 with justification**. The tool enforces this.
- Do not litigate frequentist-versus-Bayesian philosophy in the rule. Simply state: *the two numbers answer two different questions; record both.*

IV.9 — Risks

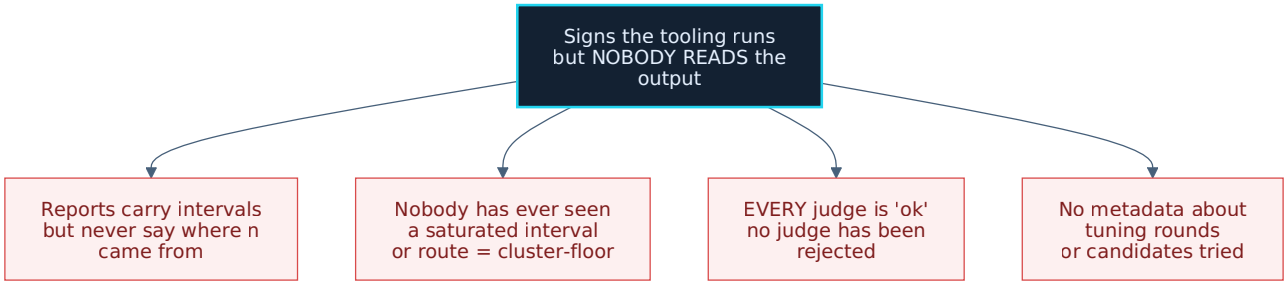
Risk	Level	Note
Cargo cult: pasting intervals into reports while never reading the checklist	**Highest**	See IV.9.1
Judge calibration skipped because it is laborious	**High**	Needs 50–100 human-scored samples. May be what kills IV.8(4).
Invented priors or excessive w	**High**	III.6.9: a wrong prior outweighs 3000 real samples
Friction: requiring k/n on every claim slows the workflow	Medium	Mitigated by the three levels in IV.3.3
Paralysis: wide intervals → nobody dares conclude anything	Medium	A wide interval signals "run more samples", not "stop"
Too many tables, wrong tool chosen	Medium	III.4.1 has a selection table; IV.3.1 has an ordered checklist

IV.9.1. The biggest risk, stated plainly

This entire document could become a ritual.

How that happens: someone pastes an interval into a report, sees a number that looks scientific, and **never reads IV.3.1**. The harness trades one form of blind confidence for another — this one harder to argue with, because it has a formula.

Four signs you have fallen into the trap:



The only viable defence I can see: make the tool **speak up when something is wrong** rather than merely printing numbers. Specifically — `wilson.py` must print an explicit warning when:

- the interval touches a boundary (saturation)
- the route falls to `cluster-floor`
- the judge is `rejected` or `biased`
- a prior was used (with its w and source)

A silent number is easy to ignore. A line reading `WARNING: judge biased toward passing` is not.

IV.10 — Open questions

Q1 — `[OBSERVED]` with an interval: hard gate or nudge? III.1.11 shows the method choice almost never changes a gate decision, so `wilson` suffices for a hard gate. But a hard gate creates friction on non-rate claims. **Unresolved.**

Q2 — Calibrate a judge once per model+prompt, or per use? At n_calib ≥ 200 "per use" is economically impossible. Caching by (model, prompt-hash) with an expiry is nearly mandatory. **How long the expiry should be — unresolved.**

Q5 — Is bootstrap / profile likelihood worth it for the combined judge interval? The theoretical floor is 1.09× naive; Bonferroni currently reaches 1.99×. Nearly half the gap remains. **Unresolved.**

Q6 — The cluster floor needs a "what counts as a passing cluster" threshold. Where does it come from? This is the most worrying one: it could recreate exactly the problem the cluster floor exists to avoid — a new invented parameter replacing the old invented ICC constant. **Unresolved.**

Q7 — Should Clopper-Pearson be the default specifically for all-pass? III.1.11 shows it demands the fewest samples at `k = n`, which is the most common probe-first case. **Unresolved.**

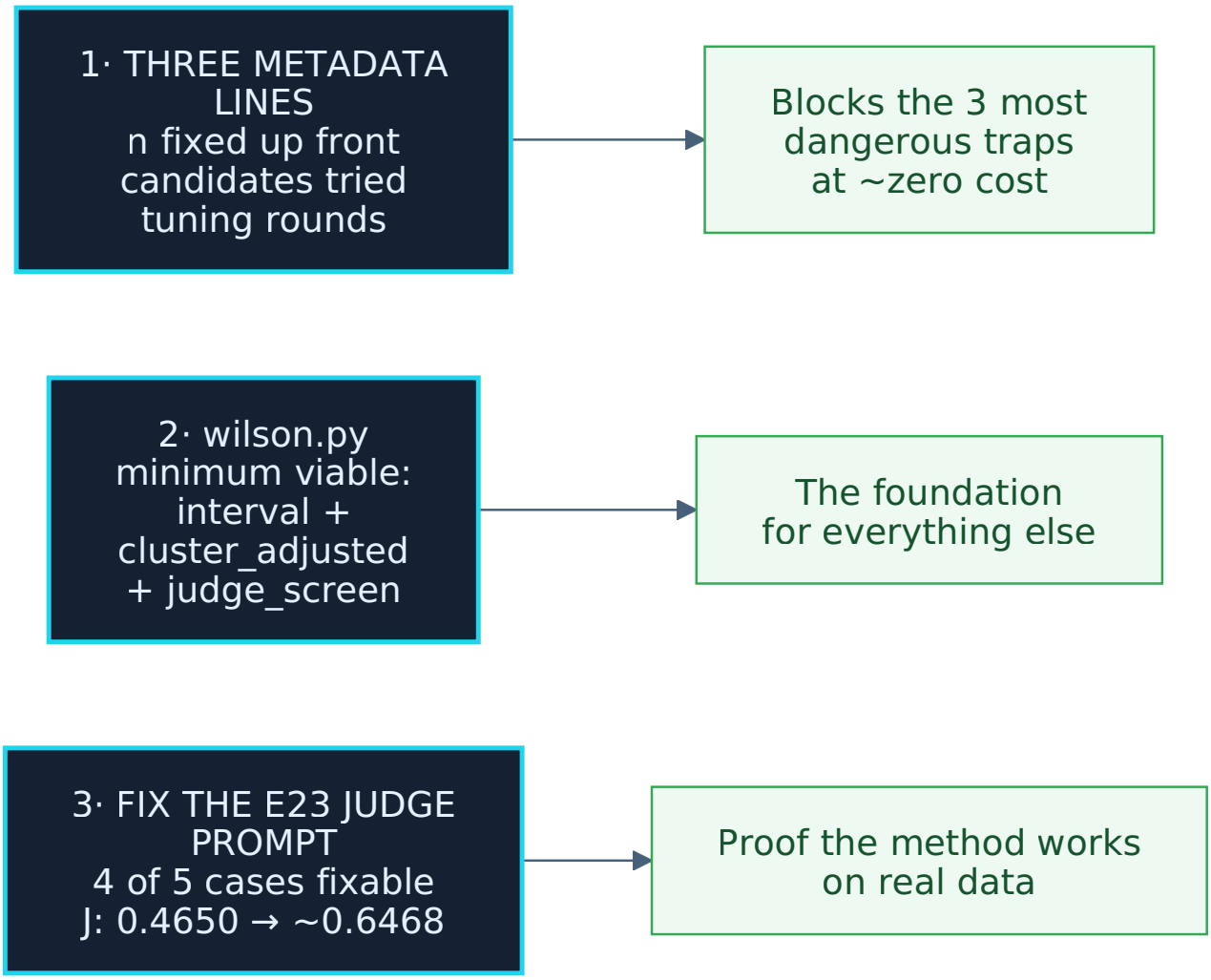
Q8 — How many rounds before a golden set expires? III.5.3 shows ~0.10 inflation after 5 rounds. But "rounds" are hard to count automatically. **Unresolved.**

Q9 — Where does the prior weight table (III.6.10) come from? The four levels 0.5 / 0.2 / 0.1 / 0 are a judgement call, not derived from data. It occupies exactly the position that "default ICC = 0.3" held before being refuted. **Unresolved.**

Q10 — Should E23 be re-run after fixing the judge prompt? *(new)* IV.4.3 estimates J rising to ~0.6468 if 4 of 5 false rejections are fixed. But that is a paper estimate, not yet run. **Requires a real probe to confirm.**

IV.11 — Where to start

If only three things get done, do these:



The ordering has a reason: task 1 is cheapest and blocks the most; task 2 underpins everything else; task 3 proves the method works on real data before larger investment.

Glossary

Term	Meaning
alpha (α)	The accepted error share: `1 − confidence`. At 95%, α = 0.05.
ANOVA estimator	Computing ICC from real data by decomposing variance into between- and within-cluster parts.
Bayesian	The approach that speaks about **belief given this specific data**. Opposed to frequentist.
Bernoulli trial	A trial with exactly two outcomes (success/failure). Each LLM call scored pass/fail is one.
Beta	The distribution used to describe belief about a rate in the Bayesian approach.
Bonferroni	Combining several uncertainty sources by dividing the error rate among them, each taking a tighter interval. Always conservative.
cluster	A group of samples sharing a source (e.g. pages from one scanned case file). Samples within a cluster are not independent.
cluster-level floor	The most conservative treatment of clustered samples: count each cluster as one sample. Equivalent to assuming ICC = 1.
Clopper-Pearson	The "exact" confidence interval, computed directly from the binomial. Coverage never falls below nominal.
confidence interval	The range of values within which the true rate plausibly lies, at a given confidence level.
confusion matrix	A 2×2 table of judge verdicts against truth: TP / FP / FN / TN.
continuity correction	A correction compensating for `k` moving in integer steps. Yields a wider, more conservative interval.
coverage	The rate at which intervals ACTUALLY contain the true value under repetition, versus the nominal confidence level.
deff (design effect)	How much the sample "shrinks" due to clustering: `deff = 1 + (m−1)·ICC`.
discordant	An item where two candidates disagree — the only items carrying information in a paired comparison.
FN (false negative)	The judge says WRONG but the truth is RIGHT.
FP (false positive)	The judge says RIGHT but the truth is WRONG.
frequentist	The approach that speaks about **the procedure under repetition**. Wilson belongs here.

Term	Meaning
garden of forking paths	Trying many dimensions and reporting only the interesting one. Inflates results even without intent.
golden set	A dataset with known-correct answers, used as the measuring instrument. E11 is one example.
held-out	A dataset set aside, never used for selection or tuning. Used for honest reporting.
ICC	How alike samples within a cluster are, from 0 (independent) to 1 (identical).
Jeffreys interval	The Bayesian interval for a rate, using a neutral "know nothing" prior.
judge (LLM judge)	A model used to score another model's or pipeline's output.
k	The number of successes.
lgamma	The log-gamma function, used to compute binomial coefficients without overflow.
McNemar	Comparing two candidates on the **same** test set, counting only discordant items.
Monte Carlo	Answering a question by random simulation. Its error shrinks as `1/√R`.
MSB / MSW	Between-cluster / within-cluster variation — the two components of the ICC formula.
n	The total number of trials.
n_eff	The effective sample size after cluster correction: `n / deff`.
Newcombe	Source of the continuity correction (1998) and of the difference interval formula.
optional stopping	Adding samples until the result looks good, then stopping. Multiplies the false-claim rate.
p̂ (p-hat)	The observed rate: `k/n`.
paired	Comparison on the same dataset (as opposed to two independent samples). Much more powerful.
posterior	Belief after seeing the data.
prior	Belief before seeing the new data — usually drawn from earlier evidence.
probe-first	The harness principle: run the real thing before building on a guess.
Rogan-Gladen	The formula recovering the true rate when the measuring instrument's error is known.

Term	Meaning
round-trip check	Verifying a formula by going forward then backward and checking you land where you started.
rule of three	Mental shortcut: 0 failures in n trials → the error rate could reach `3/n`.
saturation	An interval clamped at 0 or 1. Looks narrow but has actually overflowed — information lost.
sensitivity (sens)	The share of ACTUALLY-RIGHT cases the judge catches correctly.
specificity (spec)	The share of ACTUALLY-WRONG cases the judge catches correctly.
stratified	A golden set partitioned by data type, each measured separately.
TN (true negative)	The judge says WRONG and the truth is WRONG.
TP (true positive)	The judge says RIGHT and the truth is RIGHT.
Wald interval	The older, simpler confidence interval formula. Breaks at small n and near $\hat{p} = 0$ or 1.
Wilson interval	This document's primary formula. Never spills past [0,1], never collapses at k=0 or k=n.
winner's curse	Selecting the best of N candidates then reporting its score — that score is always inflated.
Youden's J	`sens + spec − 1`. How much better than guessing the judge is. `1/J` is the error amplification factor.
z / z-score	The threshold for a confidence level. $z = \Phi^{-1}(1 - \alpha/2)$.
Φ^{-1}	The inverse of the normal distribution. In Python: `NormalDist().inv_cdf(...)`.

Sources. All numbers computed with `python3` + `statistics.NormalDist`, session 2026-08-04. Coverage computed exactly over the binomial distribution. Simulations state their seed. Numbers quoted from evidence retain their original values: `docs/evidences/E11-260803-bang-dap-an-chuan.md`, `E14-260803-haiku-doan-huong.md`, `E23-260803-llm-doc-truong-va-trong-tai.md`; `harness/schemas/artifact-bakeoff-verdict.json`.